

Conectividad Meta4: Manual de configuración

Conector genérico e integración



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: 20 de enero de 2003.

Fecha de la última publicación: diciembre de 2013

Tabla de contenidos

Introducción

■ Acerca de esta guía	5
Finalidad	5
Audiencia	6

1 Visión general de la arquitectura

■ Acerca de este capítulo	7
■ La arquitectura Meta4 con el Conector genérico	8
La arquitectura	8
Interfaces	9
Interfaz de librería	9
Archivos XML: XMLMAP y XMLDEF	10
Tratamiento de conexiones	10
Ciclo de vida de la conexión	11
Identificador de conexión en clave	12

2 Desarrollo de una librería de servicio

■ Acerca de este capítulo	15
■ Conector genérico Meta4 - Interfaz de la librería de servicio	16
Convenciones de programación	16
La función Execute para invocar la librería de servicio	16
La versión de la librería de servicio	17
Métodos y parámetros de servicio	17
Estructuras y clases de datos compartidos	19
Control de versión de la interfaz de la librería	19
Tipos predefinidos	19
Estructuras de datos predefinidas	21
Interfaces exportadas	22
Argumentos de métodos: M4ServiceStruct y clases agregadas	22
Datos de conexión: M4ServiceConnection y M4GenericServiceConnection	25

■ Formato XMLDEF para métodos de la librería de servicio	30
Tipos de librería de servicio	30
Librerías específicas	30
Librerías genéricas	30
Importancia de XMLDEF	31
Sintaxis de XMLDEF	31
■ Construir y usar una librería de servicio	33

3 Apéndice: desarrollo de un ejemplo de librerías de servicio

■ Acerca de este apéndice	35
■ La librería de servicio helloworld	36
Paso 1: Crear el archivo XMLDEF	36
Construir la librería	36
Codificación de la librería	37
Compilar e instalar la librería	38
Importar la interfaz	39
Usar el servicio	40
■ Librería de servicio FTP	42
Crear el archivo XMLDEF	42
Construir la librería	43
Escribir la librería	43
Importar la interfaz	46
Usar el servicio	47

4 Apéndice: DTD de referencia, archivos relacionados y el Toolkit

■ Acerca de este apéndice	49
■ Archivo de definición o XMLDEF	51
Diseño y estructura del archivo XMLDEF.DTD completo	53
■ Archivo XMLMAP	56
■ Meta4Object de conexión	60
■ Archivo m4ssapi.hpp	64

Introducción

Acerca de esta guía

Finalidad

En esta guía se presenta la posibilidad que ofrece la tecnología de Meta4 de integrar sus productos, como Meta4 PeopleNet Ksystem, con otras aplicaciones o servicios externos, por medio de una interfaz conocida como Conector genérico de Meta4.

El Conector genérico es un componente de la arquitectura de las aplicaciones Meta4 que hace posible que accedan a servicios externos. Estos servicios pueden ser sistemas heredados (por ejemplo, CICS), aplicaciones propietarias (como SAP) o protocolos de comunicación (por ejemplo, FTP). El conector genérico se ha diseñado de modo que se evita el acoplamiento entre las interfaces del servicio externo y la aplicación Meta4 que llama al servicio. Esta independencia se consigue gracias a la definición de una interfaz genérica de conexión que se cumple en la parte Meta4, independientemente del servicio externo.

Para conseguir la integración, se debe crear una librería C/C++ (en adelante Librería de servicio), que actúa como interfaz entre la librería genérica y la librería de servicio externa, para cada servicio externo que quiera integrar.

Esta guía define todas las reglas, estructuras, archivos, interfaces de aplicación (API) y convenciones empleadas para desarrollar una Librería de servicio para un servicio externo.

Audiencia

Esta guía está dirigida principalmente a desarrolladores con fuertes conocimientos de programación en C/C++ involucrados en la implantación y el desarrollo de proyectos de integración con aplicaciones o servicios externos. Se asume igualmente que el lector tiene conocimientos de la estructura de los Meta4Object y componentes relacionados, así como de la arquitectura de la aplicación y de XML.

Visión general de la arquitectura

1

Acerca de este capítulo

En este capítulo se presenta la arquitectura de una solución de integración basada en el Conector genérico de las aplicaciones de Meta4, como Meta4 PeopleNet Ksystem, con un sistema externo, independientemente de las características de éste.

Para hacer que esta integración de aplicaciones sea posible, es necesario que se cumplan tres requisitos:

- El Servicio externo debe exportar una API cliente de programación (preferiblemente en C/C++, como las APIs de CICS o de SAP, por ejemplo). Además, la Librería cliente del Servicio externo debe estar disponible físicamente para el Servidor de aplicaciones de Meta4.
- Hay que desarrollar una nueva librería que actúe de puente o interfaz entre el Conector genérico y la Librería cliente del Servicio externo. El desarrollo de esta librería cliente (en adelante Librería de servicio) requiere un entorno de desarrollo C/C++ y conocimientos de programación.
- Cuando la Librería de servicio esté lista y su interfaz se haya definido en un archivo XML, es necesario enviar este archivo a la Herramienta de integración (EAI Tool), que finalmente permitirá a cualquier Meta4Object hacer llamadas a estos Métodos de servicio.

La arquitectura Meta4 con el Conector genérico

En términos generales, el Conector genérico actúa como intermediario entre aplicaciones Meta4 y servicios externos.

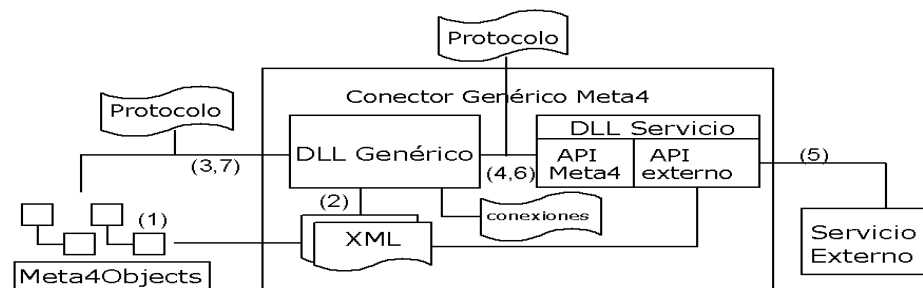
La flexibilidad de su arquitectura se consigue gracias a que un Meta4Object se puede usar como interfaz con los componentes de conexión, y el servicio externo usa unas estructuras bien definidas que son independientes de la aplicación Meta4.

La arquitectura

La arquitectura del Conector genérico usa una librería genérica y una librería de servicio. Meta4 ofrece la Librería genérica e indica cómo desarrollar la Librería de servicio que actuará como interfaz con el servicio externo. Meta4 también especifica cómo exportar los métodos de servicio que estarán disponibles desde los Meta4Objects.

Típicamente, estos Meta4Objects incluirán un Nodo desde el que se accederá al servicio, en adelante Nodo de interfaz. El Nodo de interfaz contendrá elementos de tipo método que corresponderán con métodos C++ en la Librería de servicio. Los parámetros de entrada y de salida en los métodos de la librería se asociarán con estructuras de datos en el Meta4Object (registros, elementos, etcétera).

Figura 1. Visión tecnológica



La secuencia de ejecución de una conexión con el servicio externo es la siguiente:

1. Al invocar a los métodos del Nodo de interfaz, se hace una llamada al punto de entrada del Conector genérico.
2. El Conector genérico resuelve la correspondencia entre el Meta4Object y la librería de servicio mediante un fichero (el Mapa XML).
3. El Conector genérico Meta4 convierte los argumentos de entrada que están en el formato propio de los Meta4Objects, a las estructuras de datos estipuladas en el Mapa XML.
4. A continuación, el Conector genérico invoca el punto de entrada correspondiente de la librería de servicio.
5. Al finalizar la ejecución, la librería de servicio envía los datos recogidos a la librería genérica.
6. Finalmente, la librería genérica convierte los argumentos de salida al formato propio de los Meta4Object y devuelve el control al método de Meta4Object que hizo la llamada.

Interfaces

El Conector genérico ofrece dos interfaces:

- Una interfaz con el Meta4Object que llama a los elementos de servicio. Esta interfaz especifica los puntos de entrada y los parámetros de la librería genérica, el formato y la accesibilidad del archivo XMLMAP, y el ciclo de vida de las conexiones.
- Una interfaz con la librería de servicio. Esta interfaz especifica los tipos y estructuras de datos comunes, los puntos de entrada y los parámetros de la librería de servicio, entre otras características. Tenga en cuenta que la librería de servicio es libre de implementar el servicio de cualquier forma necesaria, sea para conectarse con otros servidores o aplicaciones, o sea en su forma de cachear los datos, por ejemplo.

Interfaz de librería

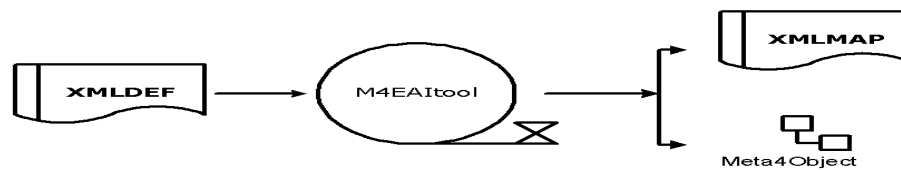
La interfaz entre el Conector genérico Meta4 y la librería de servicio obliga a la librería de servicio exportar al menos una función, o punto de entrada, llamada `Execute`. Su prototipo está incluido en el archivo `m4ssapi.hpp` (véase el apéndice). Tanto este archivo como la librería que implementa estas interfaces (`m4servicestruct`) son necesarios para compilar una librería de servicio nueva.

Archivos XML: XMLMAP y XMLDEF

La API exportada por el servicio externo se describe en un archivo XML llamado XMLDEF (XML DEFinition) con sintaxis XML definida por Meta4. El desarrollador debe escribir este archivo manualmente mediante un editor de texto y debe asegurarse de que tenga el formato adecuado. El API cliente del servicio externo determina el contenido de este archivo.

Otro archivo XML, llamado XMLMAP (XML MAPping), define la correspondencia entre el servicio externo y el Meta4Object cliente que usa este servicio externo. La herramienta de integración EAI Tool genera este nuevo archivo y el Meta4Object asociado, después de que se haya escrito el archivo XMLDEF con el formato adecuado.

Figura 2. El papel de los archivos XMLDEF y XMLMAP



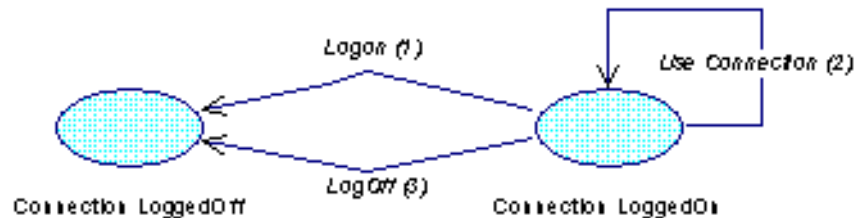
Tratamiento de conexiones

Los servicios externos pueden estar orientados al servicio (como HTTP: no se necesita conexión) u orientados a la conexión (como FTP: es necesario mantener una conexión). La mayoría de las aplicaciones son ejemplos de servicios externos orientados a la conexión, ya que requieren autenticación y necesitan que el cliente esté conectado mientras se usa el servicio.

Las librerías de servicio orientadas a la conexión deben seguir determinadas reglas en cuanto al ciclo de vida y a la gestión de las conexiones. La secuencia normal para el ciclo de vida de las conexiones en este tipo de servicio es la siguiente:

- Fase 1: Establecer la conexión (Logon)
- Fase 2: Usar la conexión para acceder a los servicios
- Fase 3: Cerrar la conexión (Logoff)

Figura 3. Ciclo de vida de la conexión



Puesto que la ruta de control de la ejecución es externa a la librería de servicio, el que realiza la llamada (el Conector genérico) debe almacenar la conexión cuando ésta se haya creado, y pasarla a las siguientes llamadas. En cada método de la librería de servicio que se invoque, existe un argumento contenedor de conexiones que sirve como argumento de salida en la fase de Logon, y como argumento de entrada en todos los demás métodos.

Así, la librería de servicio debe hacer lo siguiente:

- Crear y devolver una conexión en el método, o los métodos de Logon.
- Usar la conexión recibida en todos los demás métodos.
- Comprobar la coherencia interna de la conexión, controlando los errores cuando falta una conexión esperada o viceversa. Tenga en cuenta que la librería de servicio debe controlar el estado real de la conexión; ningún módulo externo puede usar la conexión directamente.

Ciclo de vida de la conexión

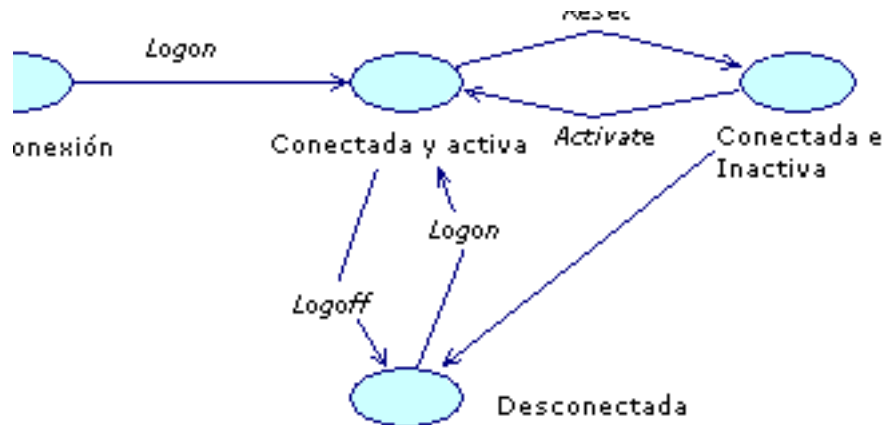
Este apartado describe un ciclo de vida completo, disponible para la implementación. En una implementación particular de una librerías de servicio orientada a conexión, no es necesario tener en consideración todas las fases del ciclo de vida propuesto; puede ocuparse únicamente de determinadas fases, según la naturaleza de los requisitos del servicio con el que conectan.

Las conexiones pueden encontrarse en tres estados:

1. **Desconectada (Logged Off)**: la conexión no está establecida; es igual a no tener ninguna conexión.
2. **Conectada y activa (Logged On And Active)**: la conexión está establecida y en uso; puede estar en un estado específico para cierta transacción o un usuario determinado.

- 3. Conectada e inactiva (*Logged On And Inactive*):** la conexión está establecida, pero no se usa. En este estado, no puede existir ninguna transacción ni estado de usuario. Este estado es muy útil en cuanto al rendimiento: en vez de desconectar y destruir la conexión, ésta se puede almacenar (*reset*) para su uso posterior. Tenga en cuenta que restablecer (*activate*) la conexión debe garantizar que se encuentre en el mismo estado que cuando se utilizó por primera vez.

Figura 4. El ciclo de vida completo de la conexión



Identificador de conexión en clave

Las conexiones a un servicio externo pueden sufrir time-outs de inactividad; el servicio externo puede desconectar la librería cliente si no se detecta actividad alguna del cliente. Estas desconexiones deben ser transparentes para el usuario de la librería de servicio.

Para volver a conectarse, una vez que la librería de servicio se haya logado correctamente, debe almacenar todos los datos relevantes de la conexión para que estén disponibles para futuras conexiones.

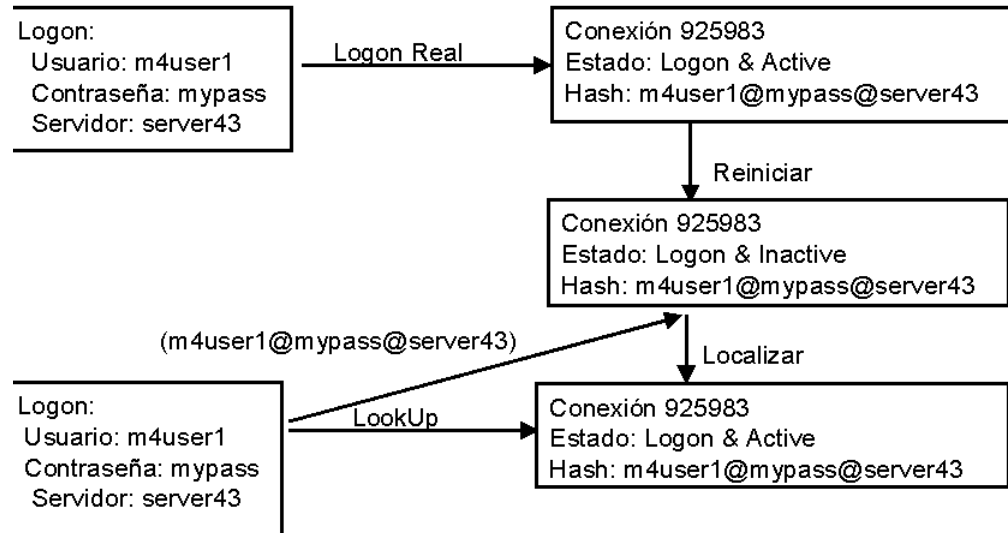
Un identificador de conexión en clave (*hashed*) es una cadena que incluye todos los parámetros de conexión pertinentes (que es posible cifrar para mayor seguridad). El identificador de conexión en clave debe construirse de tal manera que existe una relación unívoca con los parámetros del logon. No existen restricciones sobre el algoritmo y el método de cifrado opcional usado para encriptar la cadena de conexión.

Si la librería de servicio no necesita volver a conectarse para conexiones Conectadas e Inactivas, el identificador de conexión puede ser NULO.

La figura que aparece a continuación indica cómo puede funcionar la

reconexión mediante un algoritmo hash muy simple (usuario, contraseña y servidor están concatenados y separados por el signo @).

Figura 5. Nuevas conexiones con identificadores de conexión en clave



Desarrollo de una librería de servicio

2

Acerca de este capítulo

Este capítulo describe la forma de desarrollar una librería de servicio que se emplea en el Conector genérico de Meta4 para conectar aplicaciones Meta4, como Meta4 PeopleNet Ksystem, con aplicaciones externas.

La interfaz entre el Conector genérico de Meta4 y una librería de servicio define los siguientes aspectos:

- Convenciones de programación
- Versión de la librería de servicio
- Invocación de la librería de servicio
- Métodos y parámetros de servicio
- Soporte a conexiones
- Estructuras y clases de datos compartidos

Conector genérico Meta4 - Interfaz de la librería de servicio

Convenciones de programación

Todas las APIs descritos en este capítulo emplean la sintaxis de C++. Las estructuras y los tipos de datos compartidos se exportan como clases y tipos C++.

En las secciones que aparecen a continuación, el código C++ (interfaces de clase, cabeceras y ejemplos) respeta las convenciones de programación de Meta4 para C++. Las convenciones y los tipos se usan principalmente por motivos de uniformidad y un mantenimiento más sencillo del código. Algunas de estas convenciones describen:

- Cómo usar tipos básicos de C++ para que las librerías de servicio no necesiten conocer los tipos básicos de C++ de Meta4.
- Cómo usar las convenciones de nomenclatura para los identificadores: el prefijo `ai_` y `ao_` para argumentos de entrada y salida, el prefijo `m_` para miembros, entre otros.

La función **Execute** para invocar la librería de servicio

Una librería de servicio debe exportar un punto de entrada o función con el siguiente prototipo (declarado en el archivo `m4ssapi.hpp`):

```
M4SSReturnType Execute(
    M4ServiceStruct*          aio_poSSt,
    M4GenericServiceConnection* ai_pConn);
```

Todos los accesos al servicio deben invocarse mediante la función **Execute**.

El primer parámetro, **aio_poSSt**, contiene los datos de ejecución, como el nombre del método que se va a ejecutar, definición de parámetros, datos de entrada, campos de salida o campos para la gestión de errores.

El segundo parámetro, **ai_pConn**, es el contenedor para la conexión de servicio de la transacción, si existe. Este parámetro siempre contiene el mismo objeto para todas las llamadas a la transacción. Por ejemplo, típicamente se emplea para pasar al método de autenticación en el servicio externo y luego a los métodos que requieran una conexión activa.



Observe que la librería de servicio exporta únicamente un punto de entrada de ejecución, mientras, internamente, puede multiplexar la ejecución según el identificador de método que se pase.

La versión de la librería de servicio

Como cualquier método *Execute* de una librería de servicio recibe objetos definidos en la librería `m4servicestruct.dll`, es necesario comprobar la coherencia entre las distintas versiones de `m4servicestruct.dll` en tiempo de ejecución.

Cada librería de servicio debe exportar una función que devuelve su versión. Existe un prototipo de esta función definido en el archivo `m4ssapi.hpp`, y siempre hay que definir su cuerpo de la forma descrita a continuación:

```
#include "m4ssapi.hpp"
double GetInterfaceVersion(void)
{
    return (M4_SERVICE_STRUCT_DLL_INTERFACE_VERSION);
}
```

La primera vez que se carga la librería de servicio, el conector genérico hace una llamada a esta función para verificar que se ha compilado con la misma versión de la interfaz. Si no es la misma, no se invocará a ningún método de la librería de servicio, y se devuelve un error.

Métodos y parámetros de servicio

La función *Execute* de la librería de servicio es el punto de entrada/salida para todas las llamadas externas a la librería. Esta función debe comprobar el identificador del método que se pasa y tomar las medidas necesarias para ejecutar este método con los argumentos correspondientes.

Para describir los argumentos del método de servicio, hay que disponer del archivo de definición XML (XMLDEF). Este archivo se usa explícitamente para definir y documentar cada método y el conjunto de sus parámetros, en un formato reconocido por la librería de servicio. Tanto la definición XMLDEF inicial como la implementación de la librería de servicio deben coincidir, es decir, el formato XMLDEF inicial debe reflejarse en la implementación de la librería de servicio.

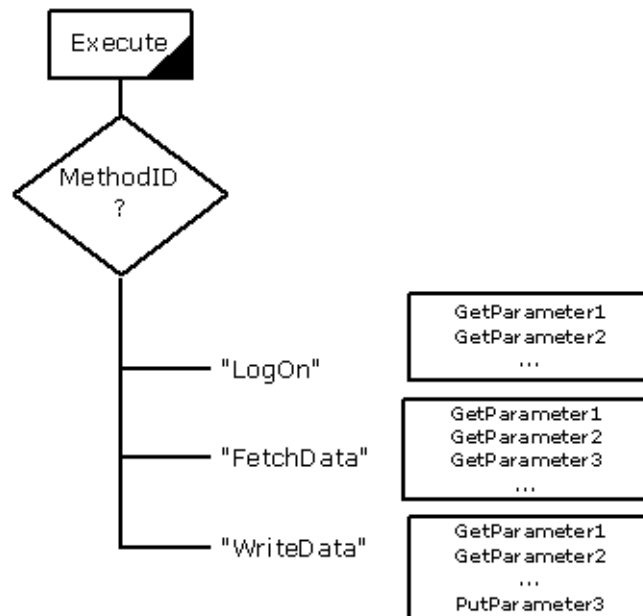
Todas las librerías de servicio debe ser programadas con controles de integridad y coherencia de datos integrados. Nunca debe presuponer la integridad de los datos recibidos. Por ejemplo, si una librería de servicio prevé tres parámetros para un método dado, hay que comprobar el número de parámetros recibidos durante la ejecución, y tomar las acciones correctivas o de notificación de errores correspondientes si la entrada no es correcta.

Todas las estructuras de datos para tratar los métodos y parámetros están incluidas en la clase M4ServiceStruct.

I

Librería de servicio: samplesvc.dll			
Método	Parametros		
	Name	Tipo	Modo
LogOn	usuario	cadena	In
	contraseña	cadena	In
FetchData	tabla	cadena	In
	columna	cadena	In
	datos	cadena	Out
WriteData	tabla	cadena	In
	columna	cadena	In
	datos	cadena	In

Figura 6. ISequencia de ejecución en la librería de servicio, samplesvc.dll



Estructuras y clases de datos compartidos

Como se ha visto anteriormente, las estructuras de datos compartidas son clave en la interfaz entre el Conector genérico Meta4 y las librerías de servicio. Para poder importar todas las clases externas necesarias, solo hay que enlazar (*link*) las librerías de servicio con la librería `m4servicestruct`. Para esto, se necesita los siguientes archivos:

- **m4ssapi.hpp**: contiene las definiciones de los tipos básicos como `M4SSreturnType`, las constantes empleadas, etcétera.
- librería **m4servicestruct**: archivo binario para las clases exportadas.

Control de versión de la interfaz de la librería

La versión de la interfaz representa la versión de todos los tipos predefinidos y estructuras de datos definidos en `m4ssapi.hpp`.

Este es el valor usado por el conector genérico para verificar la coherencia del enlace de la librería de servicio.

EJEMPLO VERSIÓN DE LA INTERFAZ DE LA LIBRERÍA

```
#define M4_SERVICE_STRUCT_DLL_INTERFACE_VERSION 4.1
```

Tipos predefinidos

Los tipos predefinidos son un conjunto de valores de enumeración para los distintos tipos de valores que se puede pasar hacia y desde la librería de servicio, como tipo de retorno, tipo de parámetro, entre otros.

EJEMPLO

```
class M4Param ;
class M4TableColumn ;

/*
 * Vector de columna
 */
typedef vector<M4TableColumn*> M4SSColumnVector ;

/*
 * Tipos de datos
 */
typedef enum {
    m4ssdt_null = 0,
```

```

        m4ssdt_number      = 1,
        m4ssdt_string      = 2,
        m4ssdt_date        = 3,
        m4ssdt_complex      = 4
    } M4SSDataType;

    /*
     * Tipos de retorno
     */
    typedef enum {

        m4ssrt_error        = -1,
        m4ssrt_success      = 0

    } M4SSReturntype;

    /*
     * Severidad del error
     */
    typedef enum {

        m4sses_error        = -1,
        m4sses_info         = 0,
        m4sses_warning      = 1

    } M4SSErrorSeverity;

    /*
     * Modos de parámetros
     */
    typedef enum {

        m4sspm_in           = 1,
        m4sspm_out          = 2,
        m4sspm_inout        = 3,
        m4sspm_ret          = 4,
        m4sspm_property     = 5

    } M4SSParameterMode;

    /*
     * Tipos de parámetros
     */
    typedef enum {

        m4sspt_tab          = 1,          // table (array of struct)
        m4sspt_str          = 2,          // string
        m4sspt_bin          = 3,          // binary
        m4sspt_fil          = 4,          // file
        m4sspt_dat          = 5,          // date
        m4sspt_tim          = 6,          // time
        m4sspt_i1           = 7,          // integer 1
        m4sspt_i2           = 8,          // integer 2
        m4sspt_i4           = 9,          // integer 4
        m4sspt_i8           = 10,         // integer 8
        m4sspt_u1           = 11,         // unsigned 1
        m4sspt_u2           = 12,         // unsigned 2
        m4sspt_u4           = 13,         // unsigned 4
        m4sspt_u8           = 14,         // unsigned 8
        m4sspt_flo          = 15,         // float
        m4sspt_dou          = 16,         // double
        m4sspt_boa          = 17,         // boolean
        m4sspt_arr          = 18,         // array
        m4sspt_struct       = 19,         // structure

    } M4SSParameterType;

```

Estructuras de datos predefinidas

La única estructura de datos predefinida es Variant, que recoge distintos tipos de datos en distintos momentos y en distintas situaciones. Se define de la siguiente forma:

EJEMPLO: ESTRUCTURAS DE DATOS PREDEFINIDAS

```
/*
 * Variant
 */
typedef struct {

    M4SSDataType    Type ;

    union {

        double      Number ;
        char*       String ;

        struct {

            short    Year ;
            char     Month ;
            char     Day ;
            char     Hour ;
            char     Minute ;
            char     Second ;

        }           Date ;

        M4Param*    Complex ;

    } Data ;

} M4SSVariant;
```

Ofrece un miembro Type para recoger el tipo de datos actual. Puede tomar un valor de enumeración, **M4SSDataType**. Cuando la estructura recoja:

- datos nulos (m4ssdt_null), el miembro Type debe ser nulo, y todos los demás miembros de la estructura se quedan sin definir.
- información numérica (m4ssdt_number), el miembro Number mantiene su valor numérico, y todos los demás miembros se quedan sin definir.
- información de cadena (m4ssdt_string), el miembro String mantiene un puntero en la parte de la memoria que recoge la cadena, y todos los demás miembros se quedan sin definir.
- información de fecha (m4ssdt_date), el miembro Date mantiene la estructura que representa la fecha, especificando el año, mes, día, hora, minuto y segundo, y todos los demás miembros se quedan sin definir.

EJEMPLO: DE CÓMO SE USAN LOS TIPOS DE DATOS

```
M4SSVariant vVariant;
```

```
// Para crear un valor de tipo nulo
vVariant.Type = m4ssdt_null;

// Para crear un valor de tipo numérico
vVariant.Type = m4ssdt_number;
vVariant.Data.Number = 126.25;

// Para crear un valor de tipo cadena
char str[ 255 ] = "mi cadena" ;
vVariant.Type = m4ssdt_string;
vVariant.Data.String = str;

// Para crear un valor de tipo fecha
vVariant.Type = m4ssdt_date;
vVariant.Data.Date.Year = 2001;
vVariant.Data.Date.Month = 12;
vVariant.Data.Date.Day = 21;
vVariant.Data.Date.Hour = 10;
vVariant.Data.Date.Minute = 0;
vVariant.Data.Date.Second = 0;
```

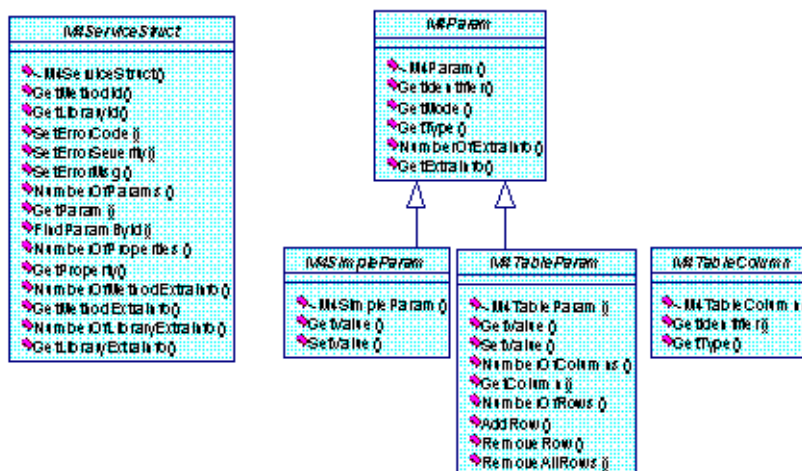
Interfaces exportadas

Para permitir la comunicación entre el conector genérico y la librería de servicio, se definen algunas interfaces de clase en la librería M4ServiceStruct para recoger los argumentos de los métodos y los datos de la conexión.

Argumentos de métodos: M4ServiceStruct y clases agregadas

Se ofrece un conjunto de clases para trabajar con los argumentos.

Figura 7. M4ServiceStruct y clases agregadas



M4ServiceStruct

Esta clase recoge toda la información usada para hacer llamadas al servicio externo. Use los métodos en este objeto para obtener información sobre la propia llamada, por ejemplo:

- Identificación de la librería
- Identificación del método
- Propiedades de la ejecución
- Parámetros de la llamada

Esta clase también ofrece métodos para notificar los errores producidos al ejecutar la llamada. En caso de producirse un error, le permite notificar los detalles del mismo mediante el mensaje de error, el código del error y la severidad del error en algunos de los elementos de error a los que accede el **Meta4Object** que hace la llamada. También le facilita información adicional a nivel de la librería de servicio.

M4Param

Esta clase recoge información sobre los parámetros de entrada y de salida. Todos los parámetros de entrada y de salida se pasan para la ejecución del método, pero únicamente los parámetros de entrada tienen un valor cuando se pasan. Puede obtener una instancia desde el objeto **M4ServiceStruct**.

Al obtener el objeto **M4Param**, puede inspeccionar o cambiar las propiedades de un parámetro, que son:

- Nombre: nombre del parámetro
- Tipo: que puede ser uno de los siguientes:

CÓDIGO	DESCRIPCIÓN
m4sspt_tab	Tabla
m4sspt_str	Cadena
m4sspt_bin	Binary
m4sspt_fil	Archivo
m4sspt_dat	Fecha
m4sspt_tim	Hora
m4sspt_i1	número entero de 1 byte
m4sspt_i2	número entero de 2 bytes
m4sspt_i4	número entero de 4 bytes

CÓDIGO	DESCRIPCIÓN
m4sspt_i8	número entero de 8 bytes
m4sspt_u1	número entero de 1 byte sin signo de puntuación
m4sspt_u2	número entero de 2 bytes sin signo de puntuación
m4sspt_u4	número entero de 4 bytes sin signo de puntuación
m4sspt_u8	número entero de 8 bytes sin signo de puntuación
m4sspt_flo	Float
m4sspt_dou	Doble
m4sspt_boo	Booleano

- Modo: uno de la lista.

CÓDIGO	DESCRIPCIÓN
m4sspm_in	Parámetro de entrada
m4sspm_out	Parámetro de salida
m4sspm_inout	Parámetro de entrada y salida
m4sspm_ret	Valor de retorno del servicio que recibe la llamada
m4sspm_property	Parámetro propiedad

También puede usar esta clase para obtener información relacionada con los parámetros directamente de la librería, si existe información adicional.

M4Param es una clase abstracta, es decir, debe convertirse en una de las clases hijas. Esto se hace recurriendo a M4SimpleParam, para parámetros con valores simples, o a M4TableParams, para parámetros con valores de tabla.

M4SimpleParam

Esta clase representa un parámetro de valor simple, que contiene un único valor. Los métodos en esta clase permiten fijar u obtener el valor del parámetro. El valor devuelto es un valor de tipo variant, **M4SSVariant**.

M4TableParam

Esta clase representa un parámetro de tipo tabla, que contiene una tabla de

valores. Los métodos en esta clase le permiten añadir o quitar filas en la tabla, además de fijar u obtener cualquier valor en estas filas. Referencie cada valor mediante los números de fila y columna. Los valores devueltos son valores de variant, **M4SSVariant**.

Puede obtener el número de columnas e información sobre todas las columnas en la tabla mediante los objetos **M4TableColumn**.

M4TableColumn

Esta clase recoge toda la información sobre una columna.

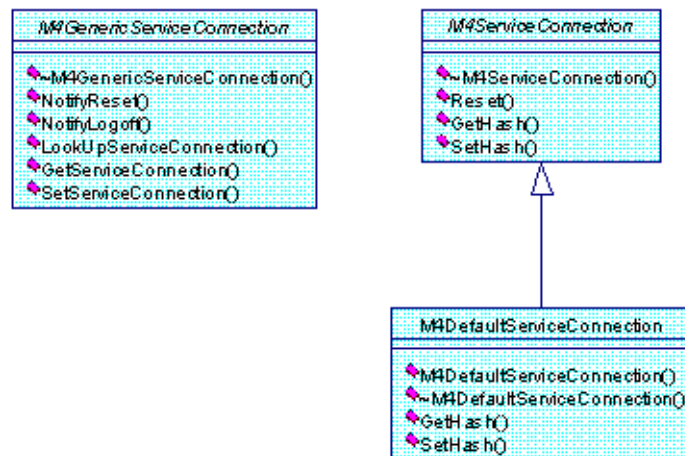
Use los métodos contenidos en esta clase para determinar:

- El identificador de la columna
- El tipo de valores que recoge esta columna (uno de los tipos indicados para parámetros simples)

Datos de conexión: M4ServiceConnection y M4GenericServiceConnection

Se ofrecen las clases **M4ServiceConnection** y **M4GenericServiceConnection** para gestionar las conexiones.

Figura 8. Clases de conexión



Las clases de conexión específicas de las librerías de servicio deben heredar de **M4GenericConnection**, que, básicamente, ofrece la interfaz que una conexión que debe implementar. No obstante, no impone ninguna restricción a la implementación interna de la conexión a través de la API externa.

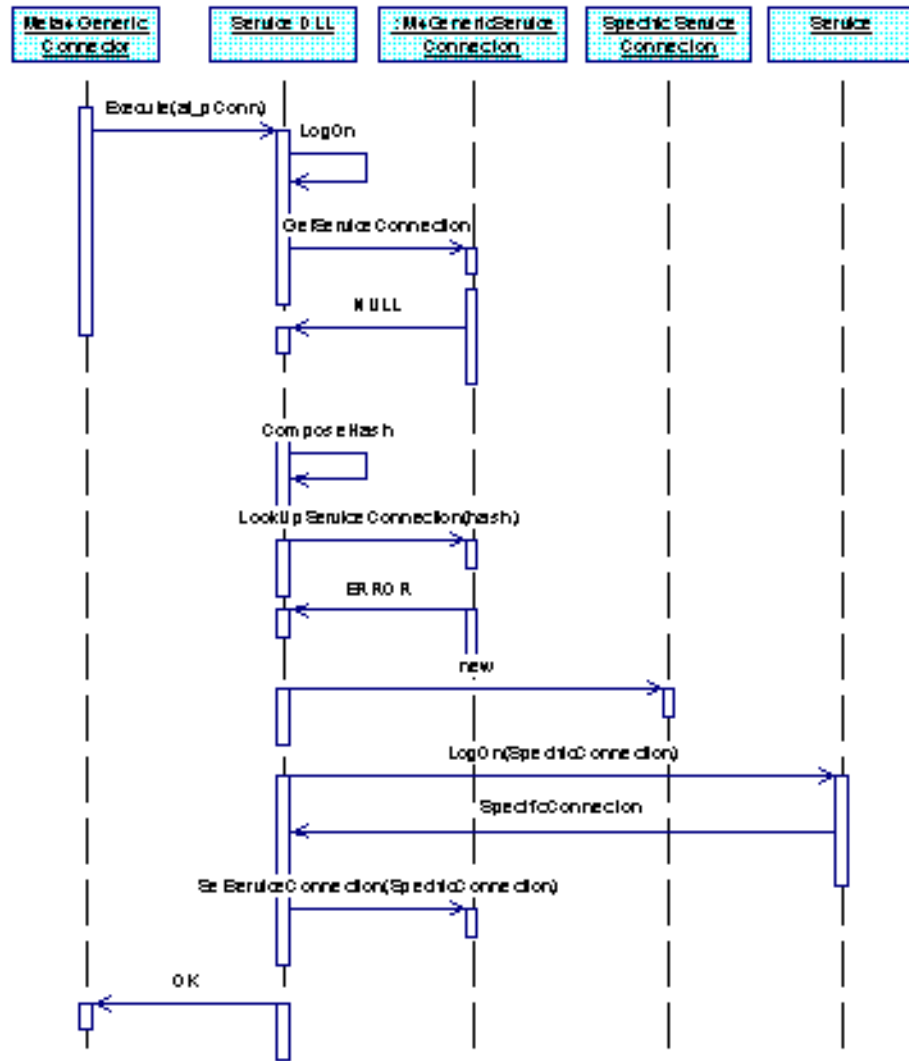
El parámetro constructor, **ai_pcHash**, puede ser NULL si el servicio no soporta conexiones Conectadas e Inactivas.

M4GenericServiceConnection es la clase referida en el segundo parámetro,

ai_pConn, de la función **Execute**. Esta clase es un contenedor para la conexión real. Cuando el servicio que se implementa no está orientado a la conexión, la librería de servicio puede simplemente obviar el parámetro, de lo contrario, la conexión debe usarse de la siguiente forma.

- **GetServiceConnection**: llame a este método antes de llamar a cualquier servicio orientado a la conexión.
 - Si no existe ninguna conexión, este método devuelve NULL. La librería de servicio debe determinar si este comportamiento es normal o si es un error. Por ejemplo, un método creado para proporcionar autenticación quizás no necesite conexión, ya que se ha diseñado para crear una nueva de todas formas.
 - Si existe una conexión, este método devuelve un puntero válido a un objeto **M4GenericServiceConnection**, que es la super-clase para la conexión real del servicio. En este caso, la librería de servicio puede asumir que es un conexión *Conectada y Activa*, aunque debe comprobar la coherencia de su estado interno.
- **SetServiceConnection**: cuando el método de una librería de servicio ha creado una nueva conexión, use este método para pasarlo al contenedor para que el Conector genérico Meta4 pueda usar la conexión en las siguientes llamadas a métodos de la librería de servicio.
- **NotifyReset y NotifyLogoff**: cuando la conexión mantenida en el objeto **M4GenericServiceConnection** cambia de estado, es necesario notificar este hecho al contenedor mediante estos métodos.
- **LookUpServiceConnection**: este método se usa únicamente cuando la librería de servicio soporta conexiones *Conectadas e Inactivas*. Si **GetServiceConnection** devuelve NULL (es decir, no existe ninguna conexión) y el método de servicio puede construir la cadena hash (normalmente en el método de autenticación), entonces **LookUpServiceConnection** devuelve SUCCESS cuando consigue encontrar una conexión *Conectadas e Inactivas* con el mismo identificador de conexión en clave; de lo contrario, devuelve ERROR. Si **LookUpServiceConnection** tiene éxito, entonces **GetServiceConnection** devuelve una conexión *Conectada y Activa* lista para usar. Normalmente, cuando devuelve éxito, eso significa que no hace falta establecer un Logon verdadero.

Figura 9. Un ejemplo de una secuencia Ejemplo de una de una secuencia Logon con un



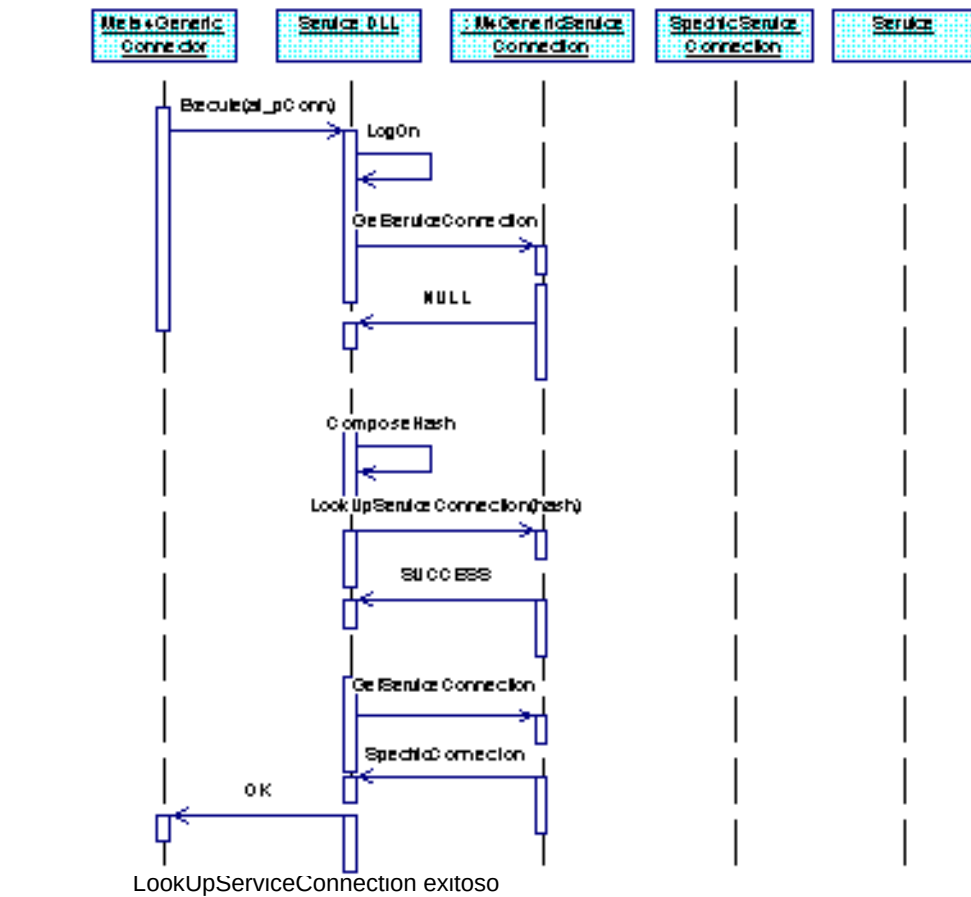
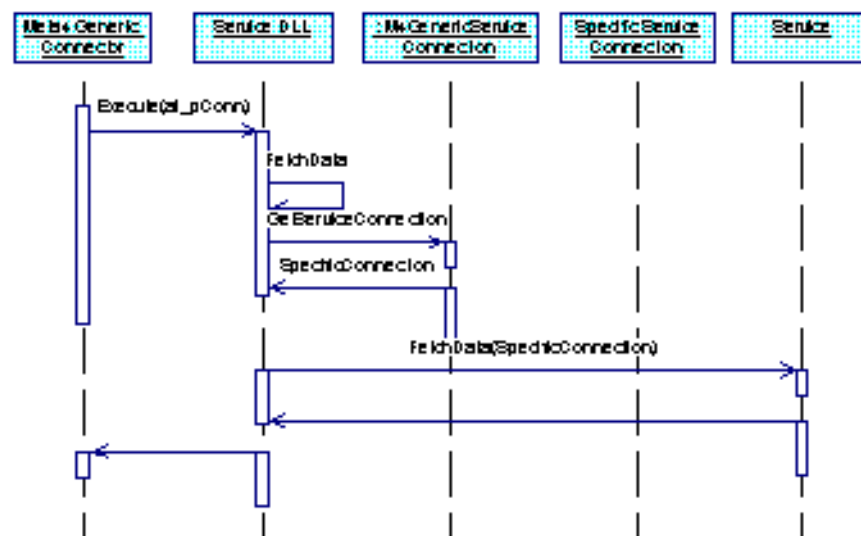


Figura 10. Ejemplo de una secuencia de método (FetchData) necesaria en una conexión



Clases exportadas

Se entrega una implementación de M4ServiceConnection por motivos de usabilidad. Esta clase es M4DefaultServiceConnection e implanta los métodos GetHash y SetHash.

Los desarrolladores encargados de crear una librería de servicio, pueden usar esta clase directamente, o pueden usarla como clase base para la implementación de clases de conexión.

Reserva de memoria

El desarrollador de una librería de servicio nunca tiene que preocuparse por reservar memoria para los parámetros de entrada y de salida y para el Conector genérico Meta4.

Cuando un parámetro que recoge un tipo Variant con una cadena como valor se pasa como argumento a la librería de servicio, **M4ServiceStruct** le reserva memoria. La librería de servicio no tiene que liberar esta memoria.

Si la librería de servicio intenta devolver un parámetro de tipo cadena, sólo tiene que poner el valor del parámetro al tipo Variant con la cadena. Cuando esto ocurre, **M4ServiceStruct** copia el valor pasado y libera el valor antiguo, si es necesario. Cuando el conector genérico lee el valor devuelto, su memoria se libera.

Sin embargo, si cuando se devuelve un parámetro de tipo cadena, la memoria fue reservada por la librería de servicio, entonces la librería de servicio se debe encargar de liberarlo.

EJEMPLO

```
M4SSVariant vVariant;

// Para asignar una cadena estática
char str[ 255 ] = "mi cadena" ;
vVariant.Type = m4ssdt_string;
vVariant.Data.String = str;

M4SimpleParam* poParam = aio_poSSt->GetParam( 0 ) ;
poParam->SetValue( &vVariant ) ;

// Para asignar una cadena dinámica
char *str = new char[ 255 ] ;
strcpy( str, "mi cadena" ) ;
vVariant.Type = m4ssdt_string;
vVariant.Data.String = str;

M4SimpleParam* poParam = aio_poSSt->GetParam( 0 ) ;
poParam->SetValue( &vVariant ) ;

// ¡¡¡No olvide liberar!!!
delete [] str ;
```

Formato XMLDEF para métodos de la librería de servicio

Cuando se diseña una librería de servicio, hay que documentar y exportar los métodos de servicio accesibles en el formato XML de Meta4 llamado XMLDEF. El formato XMLDEF especifica lo siguiente:

- **Propiedades generales del servicio** (cabecera): identificador, descripción y nombre de la librería de servicio, entre otros detalles.
- **Métodos exportados** (method): identificadores y descripciones de métodos.
- **Parámetros de método** (param): identificadores, descripciones, atributos y tipos de parámetro.

Puede haber varios archivos XMLDEF para una misma librería de servicio. Un archivo XMLDEF puede definir un solo subconjunto de los métodos accesibles por la librería de servicio. Las librerías de servicio también pueden diseñarse para que exporten un conjunto diferente de métodos, dependiendo del servidor que se use.

Tipos de librería de servicio

Básicamente, existe dos tipos de librerías de servicio que pueden ser diseñadas para usar distintos tipos de formatos XMLDEF: librerías específicas y genéricas.

Librerías específicas

Los métodos disponibles se definen mientras hace el diseño. Sus funciones se exportan en el código de la librería de servicio, por ejemplo, una librería FTP que exporta una función por cada comando posible (como dir, pwd o cd). Para este tipo de librería de servicio, existiría un XMLDEF para definir cada método posible.

Librerías genéricas

Los métodos disponibles se definen en tiempo de ejecución. Estas librerías de servicio simplemente implementan un función para acceder a los métodos del servidor. Éstos implementan una función para acceder a los métodos del servidor. En este caso, los métodos exportados se deciden en tiempo de

ejecución y dependen del servidor. For ejemplo, una librería de servicio para acceder a SAP R/3 llamaría al método necesario del servidor SAP, pero sin saber, de forma estática, cuáles son los métodos a los que puede acceder. Para este tipo de librería de servicio, existe un XMLDEF para cada servidor o tipo de servidor al que se accede.

Importancia de XMLDEF

El XMLDEF es una parte importante de la interfaz entre las librerías de servicio y el Conector genérico Meta4. El usuario final del sistema, normalmente los Meta4Objects, pueden acceder únicamente a los métodos exportados en este formato. Como el Conector genérico Meta4 actúa como intermediario entre los Meta4Objects y la librería de servicio, un Meta4Object que ejecuta un método debe conocer el identificador exacto del método, los parámetros de entrada y de salida y el identificador de la librería de servicio que debe usar, por lo que el XMLDEF es imprescindible.

Sintaxis de XMLDEF

A continuación se ofrecen fragmentos del DTD que define el formato y la sintaxis del archivo XMLDEF. Las secciones de este DTD incluyen especificaciones para el elemento raíz y todos sus nodos hijo. Véase el Apéndice: DTD de referencia y materiales relacionados para más información.

Elemento raíz:

```
<!ELEMENT m4genconnectordef (header, method+)>
<!--
  - ELEMENT: method
  - COMMENT: cada método exportado por el servicio externo.
-->
<!ELEMENT method ((param | extra)*)>
<!--
  - ATTLIST: method
  - id: CDATA #REQUIRED
  - description: CDATA #IMPLIED
  - nativeName: CDATA #IMPLIED
-->
```

Elemento header:

```
<!ELEMENT header ((property | extra)*)>
<!--
  - ATTLIST: header
  - id: CDATA #REQUIRED
  - description: CDATA #IMPLIED
  - library: CDATA #REQUIRED
  - system: CDATA #REQUIRED
  - versionDef: CDATA #REQUIRED
  - versionToolDef: CDATA #REQUIRED
  - versionService: CDATA #REQUIRED
-->
```

Elemento method:

```
<!ELEMENT method ((param | extra)*)>
```

```
<!ATTLIST method
  id CDATA #REQUIRED
  description CDATA #IMPLIED
  nativeName CDATA #IMPLIED>
```

Elemento param:

```
<!ELEMENT param (extra*, table?, extra*)>
<!ATTLIST param
  id CDATA #REQUIRED
  description CDATA #IMPLIED
  attribute (in | out | inout | return) #REQUIRED
  type (i1 | i2 | i4 | i8 | u1 | u2 | u4 | u8 | float | double | boolean
| string | binary | file | time | date | table) #REQUIRED
  nativeName CDATA #IMPLIED
>
```

Construir y usar una librería de servicio

Para conectar un Meta4Object con un servicio externo a través del Conector genérico Meta4, siga estos cuatro pasos:

1. Cree el archivo de definición XMLDEF, que contenga los métodos deseados para interactuar con la aplicación externa a través de la interfaz de servicio.
2. Construya la librería de servicio escribiendo y compilando el código C++.
3. Importe la interfaz, usando la herramienta Meta4 EAI Tool para importar el XMLDEF en el repositorio.

Esta herramienta también crea el archivo XMLMAP y construye un Meta4Object de conexión que establece una asignación entre los métodos y los parámetros de entrada/salida con la librería de servicio.

4. Utilice el Diseñador de Meta4Objects para crear un nuevo Meta4Object que use el Meta4Object generado por la herramienta EAI Tool.

Para conseguir indicaciones que le guíen paso a paso a través de la construcción y el uso de una librería de servicio, véase el *Apéndice: Desarrollo de un ejemplo de librerías de servicio*, que incluye varios ejemplos que le guían a través de los distintos pasos.

Apéndice: desarrollo de un ejemplo de librerías de servicio

3

Acerca de este apéndice

Este apéndice proporciona las indicaciones necesarias para desarrollar librerías de servicio de ejemplo que hacen uso de la arquitectura del Conector genérico, y le enseña cómo usar conexiones de servicio con servicios externos.

La librería de servicio helloworld

Esta es la librería de servicio mas simple con entrada y salida.

Este servicio dispone de un método que recibe el nombre del usuario y devuelve un mensaje de bienvenida.

Esta librería no implementa, en sí misma, la conectividad con un servicio externo, pero ayuda a comprender las interfaces XML y C++ que debe cumplir cualquier librería de servicio.

Paso 1: Crear el archivo XMLDEF

Use un editor de texto para crear el archivo. Opcionalmente, use un editor de XML para validarlo contra su DTD.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!--
- Filename: xmldef.xml
- Version: 1.0
- Descripción: Archivo de definición para la librería de servicio
helloworld.
- CompanyName: Meta4 Spain, S.A.
- Copyright: (c)2004
- ProductName: Meta4 PeopleNet Ksystem
- Fecha: 2004.08.31
-->
<m4genconnectordef>

  <header id="HELLOWORLD_SAMPLE" description="Un ejemplo de hello world!"
library="helloworld" system="SAMPLE" versionDef="1.0" versionToolDef="6.0"
versionService="1.0"/>

  <method id="print" description="devuelve un mensaje de bienvenida">
    <param id="user" description="nombre de usuario" attribute="in"
type="string"/>
    <param id="message" description="el mensaje hello world" attribute="out"
type="string"/>
  </method>

</m4genconnectordef>
```

Construir la librería

El siguiente paso implica escribir el código de la librería y luego compilarlo e instalarlo.

Escriba el código fuente en el archivo **helloworld.cpp**. Por ejemplo:

Codificación de la librería

El código fuente para la librería de servicio sería el siguiente:

```
//=====
//
// (c) Copyright 2004 Meta4 Spain S.A.
// Todos los derechos reservados.
//
// Archivo:          helloworld.cpp
// Módulo:           Conector genérico Meta4
// Fecha:            2004.08.31
// Lenguaje:         C++
// Descripción:      Un ejemplo de una librería de servicio del Conector
//                  genérico.
//                  Esta librería exporta un método que imprime un mensaje de bienvenida.
// Versión:          1.0
//
//=====

#include "m4ssapi.hpp"

/*
 * prototipos para métodos internos del servicio helloworld.
 */
void _print(M4ServiceStruct* aio_poServiceStruct);

/*
 * Función de versión de la interfaz.
 * Esta función es obligatoria y se usa en tiempo de ejecución para comprobar
 * que ambas librerías (m4servicestruc.dll y helloworld.dll (del cliente))
 * se compilaron con la misma versión de la interfaz.
 * El cuerpo de esta función debe ser siempre:
 */
double GetInterfaceVersion(void)
{
    return (M4_SERVICE_STRUCT_DLL_INTERFACE_VERSION);
}

/*
 * Punto de entrada de la DLL de servicio
 * Esta función es obligatoria y constituye el punto de entrada
 * para cualquier método en la librería de servicio. */
M4SSReturnTypes Execute(M4ServiceStruct* aio_poServiceStruct,
                        M4GenericServiceConnection *ai_poConnection)
{
    // compruebe los argumentos de entrada.
    if (NULL == aio_poServiceStruct)
    {
        // Sólo en este caso devuelve m4ssrt_error. Si no, complete los códigos
        // de error en el parámetro aio_poServiceStruct y devuelva m4ssrt_success
        return m4ssrt_error;
    }

    // limpiar errores de otras llamadas al método Execute.
    aio_poServiceStruct->SetErrorCode(0);
    aio_poServiceStruct->SetErrorSeverity(m4sses_info);
    aio_poServiceStruct->SetErrorMsg("");

    // analizar el id método.
    const char* pcMethodId = aio_poServiceStruct->GetMethodId();
    if(!strcmp(pcMethodId, "print"))
    {
        _print(aio_poServiceStruct);
    }
    else
    {
        aio_poServiceStruct->SetErrorCode(1); /* debería definir sus propios
        códigos de error */
    }
}
```

```

        aio_poServiceStruct->SetErrorSeverity(m4sses_error);
        aio_poServiceStruct->SetErrorMsg("ID Método inválido");
    }

    return m4ssrt_success;
}

/*
 * lee el parámetro de entrada 'user' y escribe un mensaje de bienvenida en
 * el parámetro 'message'.
 */
void _print(M4ServiceStruct* aio_poServiceStruct)
{
    // lee el parámetro 'user'.
    // Los parámetros se leen por posición. El orden se define siempre en
    // el archivo XMLDEF, empezando por 0.
    M4SimpleParam* poUser = (M4SimpleParam*)aio_poServiceStruct->GetParam(0);
    if (NULL == poUser)
    {
        aio_poServiceStruct->SetErrorCode(2); /* debería definir sus propios
códigos de error */
        aio_poServiceStruct->SetErrorSeverity(m4sses_error);
        aio_poServiceStruct->SetErrorMsg("No puede obtener el parámetro
'user'.");
    }
    const M4SSVariant* poUserName = poUser->GetValue();
    if (NULL == poUserName->Data.String)
    {
        aio_poServiceStruct->SetErrorCode(3); /* debería definir sus propios
códigos de error */
        aio_poServiceStruct->SetErrorSeverity(m4sses_error);
        aio_poServiceStruct->SetErrorMsg("No puede leerse el parámetro 'user'.");
    }

    // recuperar el parámetro 'message'.
    M4SimpleParam* poMessage = (M4SimpleParam*)aio_poServiceStruct->
>GetParam(1);
    if (NULL == poMessage)
    {
        aio_poServiceStruct->SetErrorCode(4); /* debería definir sus propios
códigos de error */
        aio_poServiceStruct->SetErrorSeverity(m4sses_error);
        aio_poServiceStruct->SetErrorMsg("No puede obtener el parámetro
'message'.");
    }

    // preparar mensaje.
    M4SSVariant vVariant;
    char msg[255];
    sprintf (msg, "Hello world!!!, %s", poUserName);
    vVariant.Type = m4ssdt_string;
    vVariant.Data.String = msg;

    // escribir mensaje de salida.
    poMessage->SetValue(&vVariant);
}

```

Compilar e instalar la librería

Las indicaciones para compilar e instalar la librería dependen del sistema operativo.

En los sistemas operativos de Windows soportados, Meta4 soporta Microsoft Visual Studio 6.0 como entorno de desarrollo C++. Para compilar la librería:

1. Cree un nuevo proyecto de 'Win32 Dynamic-Link Library'.
2. Seleccione **Multithreaded DLL** como la librería en tiempo de ejecución en la pestaña C++ dentro del cuadro de diálogo Project Settings.
3. Entre los archivos del proyecto, incluya el archivo **m4servicestruct.lib**, disponible en el toolkit del Conector genérico Meta4 (incluido en el CD Tecnología Base).

En los sistemas operativos de UNIX soportados, los siguientes entornos han sido probados por Meta4: En Sun Solaris, el compilador de Sun CC (5.0 o superior) y En IBM AIX , el compilador de IBM xLC_r (6.0 o superior).

En UNIX, asocie su librería a la librería **libm4servicestruct.so** que se distribuye con el Servidor de aplicaciones Meta4.

Una vez construida la librería de servicio, cópiela en el siguiente directorio:

Si ejecuta el servicio desde el servidor de aplicaciones Meta4:

`M4_APP_SERVER_INSTALLATION_ROOT/lib`

Si ejecuta desde un cliente de desarrollo:

`M4_CLIENT_INSTALLATION_ROOT/bin`

Importar la interfaz

En esta fase, usará la herramienta EAI Tool para construir el archivo XMLMAP y el Meta4Object que se corresponde con el servicio.

Asegúrese de que el servidor de aplicaciones Meta4 se haya iniciado y de que la herramienta EAI Tool también se haya instalado.

Para más información sobre la instalación de la herramienta EAI Tool, en la *Guía de instalación*, véase la sección *Conector genérico de Meta4 para conectar productos de terceros* en el capítulo *Instalación e integración con productos de terceros*.

1. Abra la herramienta EAI Tool.
2. Conéctese con un nombre de usuario válido para la aplicación Meta4 y una contraseña.
3. Haga clic en el botón **Importar interfaz** o seleccione la opción **Archivo | Importar aplicación externa** del menú.
4. Indique la ruta al archivo XMLDEF, HELLOWORLD_SAMPLE, en la ventana Importar aplicación externa.

Todos los demás campos se rellenan automáticamente, y también podrá navegar por la estructura de este archivo.

5. Haga clic en el botón **Asistente de creación del Meta4Object de conexión** o seleccione la opción **Archivo | Asistente de conexión**.
6. Seleccione el identificador del servicio, HELLOWORLD_SAMPLE, y haga clic en el botón **Siguiente**.
7. Seleccione los métodos que desea importar, normalmente todos, y haga clic en el botón **Siguiente**.
8. Elija un identificador y un nombre para el Meta4Object (por ejemplo, HELLOWORLD_SERVICE_INTERFACE - HELLO WORLD SERVICE INTERFACE) y haga clic en el botón **Terminar**.

Si la casilla **Mapear configuración de entrada como propiedades** está habilitada, los argumentos de los métodos en su servicio se mapean como propiedades en el Meta4Object, de lo contrario, se mapean como argumentos en el método generado en LN4.

El Meta4Object se generará y estará listo para usar el servicio externo a través del Conector genérico de Meta4.

Usar el servicio

El siguiente paso consiste en diseñar el Meta4Object que permitirá al usuario final llamar a la librería de servicio. Desde el Diseñador de Meta4Objects, existen dos formas de realizar llamadas a los métodos del Meta4Object mencionado anteriormente.

- Crear un nuevo Meta4Object que usa llamadas de nivel 2 para invocar a los métodos de HELLOWORLD_SERVICE_INTERFACE.
- Crear un nuevo Meta4Object que herede de HELLOWORLD_SERVICE_INTERFACE.

En este ejemplo, la primera opción usada. El Meta4Object de ejemplo se ha diseñado de la siguiente forma:



El código para el método CALL_TO_PRINT es:

```

'=====
' (c) Copyright 2004 Meta4 Spain S.A.
' Todos los derechos reservados.
'
' ti:          TST_HELLOWORLD
' metodo:     CALL_TO_PRINT
  
```

```

' Fecha:                2004.08.31
' Descripción:    Usar el Meta4Object HELLOWORLD_SERVICE_INTERFACE,
'               a través de técnicas de nivel 2 para acceder al servicio
'               externo helloworld (un servicio de conector genérico).
'               HELLOWORLD_ALIAS es un alias para HELLOWORLD_SERVICE_INTERFACE
'               definido en el alias M4Object.
' =====

' Obligatorio. Crea un registro en el nodo padre y otro en
' el nodo hijo.
HELLOWORLD_ALIAS!M4_GC_INTERFACE.AddRegister()
HELLOWORLD_ALIAS!HELLOWORLD_SERVICE_INTERFACE.AddRegister()

' Obligatorio. Llamada al método SET_ID_XMLMODEL en el nodo hijo
' para fijar el XMLMAP en el nodo padre.
HELLOWORLD_ALIAS!HELLOWORLD_SERVICE_INTERFACE.SET_ID_XMLMODEL()

' ejecuta el método PRINT.
HELLOWORLD_ALIAS!HELLOWORLD_SERVICE_INTERFACE.USER = "Juan"
HELLOWORLD_ALIAS!HELLOWORLD_SERVICE_INTERFACE.PRINT()

' comprueba los códigos de error. Los códigos de error se definen en la
' librería
' de servicio helloworld.
if HELLOWORLD_ALIAS!M4_GC_INTERFACE.ERROR_SEVERITY <> 0 then
    MessageBox("HELLOWORLD", "ERROR_MSG" +
HELLOWORLD_ALIAS!M4_GC_INTERFACE.ERROR_MSG)
    return M4_ERROR
endif

' imprime la salida.
MSG = HELLOWORLD_ALIAS!HELLOWORLD_SERVICE_INTERFACE.MESSAGE
MessageBox("HELLOWORLD", MSG)

return M4_SUCCESS

```

Librería de servicio FTP

En este ejemplo se construye una librería para acceder a un servidor FTP externo. Hace uso de la librería `ftplib3.1` como librería cliente de FTP.

Este ejemplo muestra cómo usar conexiones de servicio a servicios externos.

Crear el archivo XMLDEF

En el servicio FTP se han definido tres métodos:

- Logon - abre la conexión con el servidor FTP.
- Dir - ejecuta el comando DIR de FTP.
- Logoff - cierra la conexión.

El contenido del archivo XMLDEF es el que se muestra a continuación:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!--
- Filename: xmldef.xml
- Version: 1.0
- Descripción: Archivo de definición para la librería de servicio ftp.
- CompanyName: Meta4 Spain, S.A.
- Copyright: (c)2004
- ProductName: Meta4 PeopleNet Ksystem
- Fecha: 2004.08.31
-->
<m4genconnectordef>
  <header id="FTP_SAMPLE" description="servicio FTP" library="ftp"
    system="SAMPLE" versionDef="1.0" versionToolDef="6.0"
    versionService="1.0"/>

    <method id="logon" description="Conexión a un servidor FTP.">
      <param id="server" description="Nombre del servidor." attribute="in"
        type="string"/>
      <param id="user" description="Nombre del usuario." attribute="in"
        type="string"/>
      <param id="password" description="Contraseña." attribute="in"
        type="string"/>
    </method>

    <method id="logoff" description="Desconexión del servidor FTP, la conexión
    se cierra."/>

    <method id="dir" description="Muestra una lista de archivos y
    subdirectorios en un directorio.">
      <param id="remoteFullFileNames" description="Nombres absolutos de
      archivos remotos." attribute="out" type="table">
        <table cols="1" rows="*" mode="table">
          <th id="remoteFullFileName" description="Nombres absolutos de
          archivos remotos." type="string"/>
        </table>
      </param>
    </method>
</m4genconnectordef>
```

Construir la librería

El siguiente paso implica escribir el código de la librería y luego compilarlo e instalarlo.

Escribir la librería

El código fuente para la librería de servicio es el siguiente:

```
//=====
//
// (c) Copyright 2004 Meta4 Spain S.A.
// Todos los derechos reservados.
//
// Archivo:          ftp.cpp
// Módulo:          Conector genérico Meta4
// Fecha:           2004.08.31
// Lenguaje:        C++
// Descripción:     Un ejemplo de una librería de servicio del Conector
//                  genérico.
//                  Este ejemplo muestra cómo usar conexiones de servicio desde el
//                  api del conector genérico.
// Versión:         1.0
//
//=====
...

/*
 * Punto de entrada de la DLL de servicio.
 * Esta función es obligatoria y constituye el punto de entrada
 * para cualquier método en la librería de servicio.
 */
M4SSReturnType Execute(M4ServiceStruct* aio_poServiceStruct,
                      M4GenericServiceConnection *ai_poConnection)
{
...
    // analizar el id método.
    const char* pcMethodId = aio_poServiceStruct->GetMethodId();
    if(!strcmp(pcMethodId, "login"))
    {
        _login(aio_poServiceStruct, ai_poConnection);
    }

...
    else
    {
        char buffer[1024];
        sprintf(buffer, "ID método inválido: %s.", pcMethodId);
        _seterror(aio_poServiceStruct, buffer);
    }

    return m4ssrt_success;
}

/*
 * conexión al servidor ftp.
 * parámetros:
 * 0.host:cadena
 * 1.user:cadena
 * 2.password:cadena
 */
void _login(M4ServiceStruct* aio_poServiceStruct, M4GenericServiceConnection
*ai_poConnection)
```

```

{
    // Establecer la conexión
    FTPServiceConnection* pServConn = (FTPServiceConnection*)(ai_poConnection-
>GetServiceConnection());
    if (pServConn)
    {
        _seterror(aio_poServiceStruct, "El usuario ya está conectado. Se usará
la conexión actual.");
        return;
    }

    // crear cadena Hash de conexión. Host+User+Password.
    char *pcHash = new char[strlen(pcHost) + strlen(pcUser) + strlen(pcPasswd)
+ 2 + 1];
    sprintf(pcHash, "%s@%s@%s", pcHost, pcUser, pcPasswd);

    // crear FTPServiceConnection. La clase FTPServiceConnection recoge
información específica
    // para la conexión ftp.
    pServConn = new FTPServiceConnection(pcHash);
    delete [] pcHash;

    // inicializar la librería externa ftp3.1.
    FtpInit();

    // conexión al servidor ftp.
    netbuf* pNetbuf = NULL;
    int iRes = FtpConnect(pcHost, &pNetbuf);
    if (iRes != 1)
    {
        _seterror(aio_poServiceStruct, "Could not LogOn. No se ha devuelto
ninguna conexión.");
        delete pServConn;
        return;
    }

    // guarda la información interna del servidor ftp en el objeto
FTPServiceConnection.
    pServConn->SetNetbuf(pNetbuf);

    // Conexión
    iRes = FtpLogin(pcUser, pcPasswd, pServConn->GetNetbuf());
    if (iRes != 1)
    {
        _seterror(aio_poServiceStruct, "Could not LogOn. Usuario o contraseña
inválidos.");
        delete pServConn;
        return;
    }

    // Conexión correcta. Conexión devuelta.
    ai_poConnection->SetServiceConnection(pServConn);
}

/*
* cerrar la conexión con el servidor ftp.
*/
void _logoff(M4ServiceStruct* aio_poServiceStruct,
M4GenericServiceConnection *ai_poConnection)
{
    // Establecer la conexión
    FTPServiceConnection* pSC = (FTPServiceConnection *) (ai_poConnection-
>GetServiceConnection());
    if (NULL == pSC)
    {
        _seterror(aio_poServiceStruct, "La conexión debe pasarse a este método.
El usuario no se ha conectado.");
        return;
    }
}

```

```

    // la desconexión se lleva a cabo a través del destructor
    FTPServiceConnection.
    delete pSC;
    ai_poConnection->SetServiceConnection(NULL);
    ai_poConnection->NotifyLogoff();
}

/*
 * obtener los archivos en el directorio actual.
 * parámetros:
 *     0.files:table(1)out
 */
void _dir(M4ServiceStruct* aio_poServiceStruct, M4GenericServiceConnection
*ai_poConnection)
{
    ...
    // Establecer la conexión.
    FTPServiceConnection* pSC = (FTPServiceConnection *) (ai_poConnection-
>GetServiceConnection());
    if (NULL == pSC)
    {
        _seterror(aio_poServiceStruct, "La conexión debe pasarse a este método.
El usuario no se ha conectado.");
        return;
    }
    ...
    // obtener filenames.
    const char* filename = "c:\\temp\\ftpdire.tmp";
    iRes = FtpDir(filename, pcCurrentDir, pSC->GetNetbuf());
    if (iRes != 1)
    {
        char buffer[1024];
        sprintf(buffer, "FtpDir(): error \"%s\". dir: \"%s\"",
FtpLastResponse(pSC->GetNetbuf()), pcCurrentDir);
        _seterror(aio_poServiceStruct, buffer);
        return;
    }
    return;
}

/*
 * Clase FTPServiceConnection.
 */
FTPServiceConnection::FTPServiceConnection(const char* ai_pHash)
: M4DefaultServiceConnection(ai_pHash)
{
    m_pNetbuf = NULL;
}

FTPServiceConnection::~FTPServiceConnection()
{
    {
        // Si m_pNetbuf no es NULL, se establece la conexión interna,
        // Entonces se desconecta y la conexión se destruye.
        if (NULL != m_pNetbuf)
        {
            FtpQuit(m_pNetbuf);
        }

        m_pNetbuf = NULL;
    }
}

M4SSReturnTypes FTPServiceConnection::Reset()
{
    {
        return m4ssrt_success;
    }
}

netbuf* FTPServiceConnection::GetNetbuf()
{
    {
        return m_pNetbuf;
    }
}

```

```
void FTPServiceConnection::SetNetbuf(netbuf* ai_pNetbuf)
{
    m_pNetbuf = ai_pNetbuf;
}
```

Importar la interfaz

En esta fase, usará la herramienta EAI Tool para construir el archivo XMLMAP y el Meta4Object que se corresponde con el servicio.

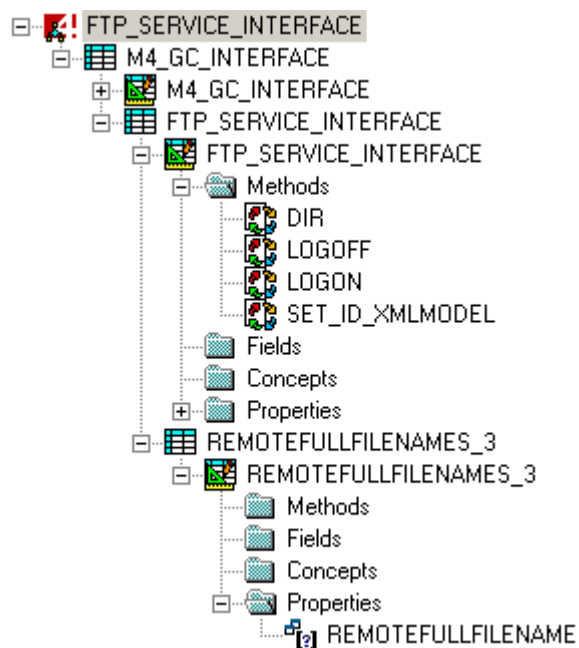
Asegúrese de que el servidor de la aplicación Meta4 se haya arrancado y de que la herramienta EAI Tool también se haya instalado. Para más información sobre la instalación de la herramienta EAI Tool, en la *Guía de instalación*, véase la sección *Conector genérico de Meta4 para conectar productos de terceros* en el capítulo *Instalación e integración con productos de terceros*.

1. Abra la herramienta EAI Tool.
2. Conéctese con un nombre de usuario válido para la aplicación Meta4 y una contraseña.
3. Haga clic en el botón **Importar interfaz** o seleccione la opción **Archivo | Importar aplicación externa** del menú.
4. Indique la ruta al archivo XMLDEF, FTP_SERVICE_INTERFACE - FTP SERVICE INTERFACE, en la ventana Importar aplicación externa.

Todos los demás campos se rellenan automáticamente, y también podrá navegar por la estructura de este archivo.
5. Haga clic en el botón **Asistente de creación del Meta4Object de conexión** o seleccione la opción **Archivo | Asistente de conexión**.
6. Seleccione el identificador de su servicio, FTP_SERVICE_INTERFACE - FTP SERVICE INTERFACE, y a continuación haga clic en el botón **Siguiente**.
7. Seleccione los métodos que desea importar, normalmente todos, y haga clic en el botón **Siguiente**.
8. Elija un identificador y un nombre para el Meta4Object (por ejemplo, FTP_SERVICE_INTERFACE - FTP SERVICE INTERFACE) y haga clic en el botón **Terminar**.

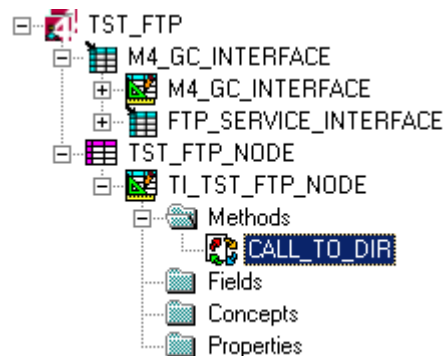
Asegúrese de que la casilla de verificación **Mapear configuración de entrada como propiedades** esté desactivada. Los parámetros simples (como cadena, numérico, etcétera) se mapean como argumentos de método LN4. Los parámetros complejos (como tablas, etcétera) se mapean siempre como nodos de Meta4Objects.

El Meta4Object se generará y estará listo para usar el servicio externo a través del Conector genérico de Meta4. El Meta4Object generado automáticamente sería el siguiente:



Usar el servicio

Ahora puede heredar este Meta4Object para crear uno nuevo que use el servicio FTP.



El código para el método CALL_TO_DIR es:

```
'=====
' (c) Copyright 2004 Meta4 Spain S.A.
' Todos los derechos reservados.
'
' ti:          TST_FTP_NODE
' metodo:      CALL_TO_DIR
' Fecha:       2004.08.31
' Descripción:  Uso del m4object FTP_SERVICE_INTERFACE m4object
'              para acceder al servicio ftp (un servicio de conector genérico).
'=====
```

```
' Obligatorio. Crea un registro en el nodo padre y otre en
' el nodo hijo.
M4_GC_INTERFACE.AddRegister()
FTP_SERVICE_INTERFACE.AddRegister()

' Obligatorio. Llamada al método SET_ID_XMLMODEL en el nodo hijo
' para fijar el XMLMAP en el nodo padre.
FTP_SERVICE_INTERFACE.SET_ID_XMLMODEL()

' ejecuta el método LOGON.
FTP_SERVICE_INTERFACE.LOGON("miriamcant.meta4.com", "anonymous", "")
if M4_GC_INTERFACE.ERROR_SEVERITY <> 0 then
    MessageBox("FTP", "LOGON ERROR:" + M4_GC_INTERFACE.ERROR_MSG)
    return M4_ERROR
endif

' ejecuta el método DIR.
FTP_SERVICE_INTERFACE.DIR()
if M4_GC_INTERFACE.ERROR_SEVERITY <> 0 then
    MessageBox("FTP", "DIR ERROR:" + M4_GC_INTERFACE.ERROR_MSG)
    return M4_ERROR
endif

' ejecuta el método LOGOFF.
FTP_SERVICE_INTERFACE.LOGOFF()
if M4_GC_INTERFACE.ERROR_SEVERITY <> 0 then
    MessageBox("FTP", "LOGOFF ERROR:" + M4_GC_INTERFACE.ERROR_MSG)
    return M4_ERROR
endif

return M4_SUCCESS
```

Apéndice: DTD de referencia, archivos relacionados y el Toolkit

4

Acerca de este apéndice

A lo largo de esta guía se hace referencia a varios archivos, que son necesarios para desarrollar y exportar los métodos de las nuevas librerías de servicio. Estos archivos se incluyen en este apéndice, pero también se distribuyen, para mayor comodidad, con el Toolkit de desarrollador de Conector genérico de Meta4, en el CD de tecnología base.

Tenga en cuenta que si desarrolla sobre la plataforma de Windows, también necesitará un archivo que sólo se distribuye con el Toolkit, el archivo **m4servicestruct.lib**. Microsoft C++ Visual Studio necesita este archivo para enlazar correctamente con **m4servicestruct.dll**.

Este apéndice incluye una descripción del contenido de los DTD usados con el Conector genérico para construir la implantación de la interfaz con una librería de servicio externa.

También incluye una descripción del Meta4Object usado para realizar la conexión entre la interfaz de la aplicación y el Conector genérico.

Además, contiene el código para el archivo **m4ssapi.hpp**.

Archivo de definición o XMLDEF

XMLDEF define los métodos que se pueden ejecutar a través de la librería de servicio que permite la comunicación con una aplicación externa.

En un archivo de definición XMLDEF se define cuáles son los métodos necesarios para interactuar con la aplicación externa. Este método se define en el apartado **m4genconnectordef**. La estructuración de la sección es la siguiente:

- **Cabecera:** contiene los siguientes atributos:
 - **ID:** código identificador que hace referencia al archivo de definición
 - **Description:** descripción del archivo (opcional)
 - **Library:** librería de servicio que implanta la conexión con la aplicación externa
 - **System:** sistema externo al que se realiza la conexión
 - **VersionDef:** versión del archivo de definición
 - **VersionToolDef:** versión de la herramienta con la que se ha creado el archivo de definición
 - **VersionService:** versión de la librería de servicio

Puede contener secciones **Property** en las que se puede configurar aspectos concretos de cada una de las librerías de servicio, como decidir el plazo de espera en establecer la conexión, el tamaño máximo de bytes recibidos, etcétera. Las secciones **Property** tienen los siguientes atributos:

- **Name:** nombre identificativo de la propiedad
- **Description:** descripción de la propiedad (opcional)
- **Type:** Tipo de datos del parámetro. Los valores posibles son:
 - **Tab:** table
 - **Str:** cadena de caracteres
 - **Bin:** cadena binaria
 - **File:** archivo
 - **Date:** fechas
 - **Tim:** hora
 - **I1:** número entero de 1 byte
 - **I2:** número entero de 2 bytes
 - **I4:** número entero de 4 bytes
 - **I8:** número entero de 8 bytes
 - **U1:** número entero de 1 byte sin signo de puntuación
 - **U2:** número entero de 2 bytes sin signo de puntuación
 - **U4:** número entero de 4 bytes sin signo de puntuación

- **U8**: número entero de 8 bytes sin signo de puntuación
- **Flo**: número decimal
- **Dou**: número decimal con mayor precisión
- **Boo**: booleano

```
<header id="CICS en Atenea" description="Sistema CICS puesto en
Atenea" library="m4cics" system="CICS" versionDef="1.0"
versionToolDef="1.0" versionService="1.0">
  <extra name="extra1" value=""/>
  <property name="prop1" description="Descripción de prop1"
type="i2"/>
  <extra name="extra2" value=""/>
</header>
```

- **Método**: existe una sección como esta para cada método que se va a ejecutar. con los siguientes atributos:

- **ID**: identificador del método
- **Description**: breve descripción del método (opcional)

También puede contener secciones **Param** que contienen los siguientes atributos:

- **ID**: identificador del parámetro
- **Description**: breve descripción del parámetro (opcional)
- **Atributo de**: puede tener uno de los siguientes valores:
 - **In**: si el parámetro es de entrada
 - **Out**: si el parámetro es de salida
 - **InOut**: en el caso de que el parámetro sea de entrada y salida
 - **Ret**: si el parámetro es de retorno del método
- **Type**: tipo de datos que contiene el parámetro (sus posibles valores se han descrito anteriormente)

Las secciones **Param** que contienen un atributo de tipo **Tabla** tienen una sección denominada **Table** que consta de los siguientes atributos:

- **Cols**: número de columnas en la tabla
- **Rows**: no tiene significado en un archivo de definición y se representa siempre con un asterisco *.

Para cada columna definida se introduce una nueva sección llamada **th** con los siguientes atributos:

- **ID**: código identificador de la columna
- **Type**: tipo de datos que contiene el parámetro. Puede tomar uno de los valores descritos anteriormente, excepto el valor **Tab**, ya que no puede existir una tabla dentro de otra.

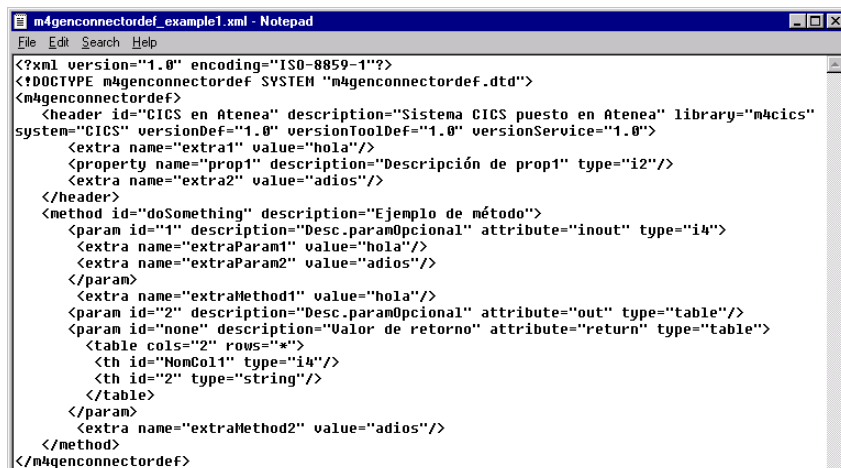
```

<method id="doSomething" description="Ejemplo de método">
  <param id="1" description="Desc.paramOpcional" attribute="inout"
    type="i4">
  </param>
  <extra name="extraMethod1" value=""/>
  <param id="2" description="Desc.paramOpcional" attribute="out"
    type="tab"/>
  <param id="none" description="" attribute="out" type="tab">
    <table cols="2" rows="*">
      <th id="NomCol1" type="i4"/>
      <th id="2" type="string"/>
    </table>
  </param>
</method>

```

Finalmente, las secciones del archivo XMLDEF pueden contener una o más secciones llamadas **Extra**. Estas secciones incluyen la información adicional que desea comunicar a la librería de servicio. Esta sección se compone de los atributos **Name** y **Value**.

Figura 11. Archivo de definición XMLDEF



Diseño y estructura del archivo XMLDEF.DTD completo

Resumiendo la información presentada anteriormente, el siguiente ejemplo muestra una posible estructura para un archivo XMLDEF.

EJEMPLO DE LA ESTRUCTURA Y DISEÑO DE UN XMLDEF.DTD

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<!--
  - Filename: xmldef.dtd

```

```

- Version: 2.0
- Description: DTD (Document Type Definition) para los archivos de
definición del Conector genérico de Meta4.
- CompanyName: Meta4 Spain, S.A.
- Copyright: (c)2004
- ProductName: Meta4Mind Set
- Fecha: 2004.08.27
-->
<!--
- ELEMENT: cabecera
- COMMENT: descripción de este servicio.
- ATTLIST:
- id: identificador del servicio
- descripción: breve descripción del servicio
- library: nombre de la librería de servicio dll (ejemplo: ftpsvc.dll)
- sistema: el sistema externo (ejemplo: ftp, sap, soap)
- versionDef: versión de esta definición (ejemplo: 1.0)
- versionToolDef: versión de Meta4 EAITool (ejemplo: 6.0). Obligatorio
aunque se usa únicamente por motivos de documentación.
- versionService: versión del sistema externo o versión del DLL de
servicio (ejemplo: 1.0)
-->
<!ELEMENT header ((property | extra)*)>
<!ATTLIST header
    id CDATA #REQUIRED
    description CDATA #IMPLIED
    library CDATA #REQUIRED
    system CDATA #REQUIRED
    versionDef CDATA #REQUIRED
    versionToolDef CDATA #REQUIRED
    versionService CDATA #REQUIRED
>
<!--
-
- ELEMENT: m4genconnectordef
- COMMENT: elemento raíz de la definición xml para el servicio.
-->
<!ELEMENT m4genconnectordef (header, method+)>
<!--
- ELEMENT: method
- COMMENT: cada método exportado por el servicio externo.
-->
<!ELEMENT method ((param | extra)*)>
<!ATTLIST method
    id CDATA #REQUIRED
    description CDATA #IMPLIED
    nativeName CDATA #IMPLIED
>
<!--
- ELEMENT: param
-->
<!ELEMENT param (extra*, table?, extra*)>
<!ATTLIST param
    id CDATA #REQUIRED
    description CDATA #IMPLIED
    attribute (in | out | inout | return) #REQUIRED
    type (i1 | i2 | i4 | i8 | u1 | u2 | u4 | u8 | float | double | boolean |
string | binary | file | time | date | table) #REQUIRED
    nativeName CDATA #IMPLIED
>
<!--

```

```

- ELEMENT: extra
- COMMENT: atributo definido por el usuario.
-->
<!ELEMENT extra EMPTY>
<!ATTLIST extra
  name CDATA #REQUIRED
  value CDATA #IMPLIED
>
<!--
- ELEMENT: propiedad
- COMMENT: atributo definido por el usuario.
-->
<!ELEMENT property EMPTY>
<!ATTLIST property
  name CDATA #REQUIRED
  description CDATA #IMPLIED
  type (i1 | i2 | i4 | i8 | u1 | u2 | u4 | u8 | float | double | boolean |
string | binary | file | time | date | table) #REQUIRED
>
<!--
- ELEMENT: table
- COMMENT:
-->
<!ELEMENT table (th+)>
<!ATTLIST table
  cols CDATA #REQUIRED
  rows CDATA #REQUIRED
  mode (array | struct | table) #REQUIRED
>
<!--
- ELEMENT: th
- COMMENT:
-->
<!ELEMENT th (table?)>
<!ATTLIST th
  id CDATA #REQUIRED
  description CDATA #IMPLIED
  type (i1 | i2 | i4 | i8 | u1 | u2 | u4 | u8 | float | double | boolean |
string | binary | file | time | date | table) #REQUIRED
  nativeName CDATA #IMPLIED
>

```

Archivo XMLMAP

Un archivo XMLMAP tiene una estructura parecida a la de un archivo XMLDEF, y su función es relacionar los métodos y parámetros de la aplicación externa con métodos LN4 y propiedades LN4 del Meta4Object de conexión que se utiliza como interfaz entre la aplicación de Meta4 y la aplicación externa.

Define los métodos para el archivo de mapeo dentro de la sección **m4genconnectormap**, que tiene la siguiente estructura:

- **Cabecera:** se compone de los atributos explicados en el archivo de definición, más los siguientes atributos nuevos:
 - **ID:** identificador del archivo de mapeo
 - **DefinitionId:** código identificador que hace referencia al archivo de definición
 - **VersionMap:** versión del archivo de mapeo
 - **VersionToolMap:** versión de la herramienta con la que se ha creado el archivo de mapeo

Las secciones **Property** que aparecen en un archivo de definición tienen una sección adicional que relaciona las propiedades con los elementos de un Meta4Object de conexión. Los disponibles son los siguientes:

- **ValueDefault:** se asigna un valor por defecto a la propiedad, con un atributo que contiene el valor por defecto.
- **ValueItem:** asocia a la propiedad un elemento de un Meta4Object. El valor del atributo tiene el formato *NODO.ITEM*. Opcionalmente, se puede usar el atributo por defecto **Default**. Este valor se usa cuando el elemento sea *Null*.
- **ValueIgnore:** indica que se no se va a tener en cuenta la propiedad.

Opcionalmente, se puede añadir una sección llamada **SystemProperty**. Esta sección tiene la misma estructura que la sección **Property**. A diferencia de **Property**, esta sección sólo afecta al Conector genérico y no a la librería de servicio. Se utiliza para configurar aspectos concretos del mismo, como gestionar el tiempo en las cachés.

```
<header id="CICS en Atenea para HR" definitionId="CICS en Atenea"
description="Sistema CICS puesto en Atenea para HR"
versionMap="1.0" versionToolMap="1.0" versionDef="1.0"
versionToolDef="1.0" versionService="1.0" library="m4cics"
system="CICS">
  <extra name="extra1" value=""/>
  <property name="timeOut" description="function calls timeout"
type="i4">
    <valueItem item="NODE.ITEM"/>
  </property>
  <systemProperty name="timeOut2" description="function calls
timeout" type="i4">
    <valueItem item="NODE.ITEM"/>
  </systemProperty>
  <extra name="extra2" value=""/>
</header>
```

- **Método:** como en el caso del archivo de definición, existe un apartado como éste para cada método disponible. Se compone de los atributos explicados en el archivo de definición más uno nuevo:

- **Name:** nombre del método.

Las secciones **Param** en un archivo de mapeo tienen una sección que asocia el valor del atributo. Aparte de las secciones **ValueDefault**, **ValueItem** y **ValueIgnore**, explicadas antes, también contiene una de las siguientes secciones:

- **ValueParam:** esta sección se asocia al parámetro de un método. El valor de su atributo **Param** de un método LN4 del Meta4Object, corresponde al número de orden del parámetro del que toma el valor. Opcionalmente, puede usar un atributo por defecto llamado **Default**. Este valor se usa si el valor del parámetro es *Null*.
- **ValueReturn:** esta sección indica que el valor del parámetro se va a devolver como resultado de la ejecución de un método LN4 del Meta4Object. Esta asociación sólo puede utilizarse con parámetros de tipo **Out**.
- **ValueTable:** esta sección indica valores por defecto para una tabla. Consta de los siguientes atributos:
 - **Cols:** número de columnas en la tabla
 - **Rows:** número de filas que posee la tabla

Para cada columna definida se introduce una nueva sección llamada **th** con los siguientes atributos:

- **ID:** código identificador de la columna

- **Type:** tipo de datos que contiene el parámetro (sus posibles valores se han descrito anteriormente en el archivo de definición)

Para cada fila de datos definida se introduce una sección denominada **tr**. Esta sección se corresponde con las columnas que tiene la tabla. Para cada columna de la tabla o sección **tr** habrá una sección **ValueDefault**.

- **NodoValores** : esta sección asocia el parámetro a un nodo de un Meta4Object. Sólo puede existir en parámetros de tipo tabla. Especifica que los datos de tipo tabla se introducen o se recuperan de un nodo y contiene los siguientes atributos:
 - **Node:** nombre del nodo
 - **RegFirstIn:** el primer valor que recoge del nodo (sólo con parámetros de entrada, opcional)
 - **RegLastIn:** el último valor que recoge el nodo (sólo con parámetros de entrada, opcional)
 - **RegCountIn:** número de registros que se quiere introducir en el nodo (sólo con parámetros de entrada, opcional)
 - **FileModeOut:** se utiliza para las tablas con valor de salida (**Out** o **InOut**) y tiene dos valores:
 - **Append:** añade los datos al nodo y borra los datos ya existentes.
 - **No append:** añade los datos al nodo y borra los datos ya existentes.

Para cada columna, se introduce una nueva sección llamada **th** con los siguientes atributos:

- **ID:** identificador del elemento
- **Type:** tipo de datos que contiene el parámetro, sus valores se han descrito anteriormente
- **Item:** elemento del nodo del que recoge los datos. El valor del atributo tiene el formato *ITEM* o *NODO.ITEM*.

```

<method id="doSomething" name="Método Ln4." description="Ejemplo de método">
  <extra name="extraMethod1" value=""/>
  <param id="none" description="Desc.paramOpcional" attribute="return"
    type="i4">
    <valueItem item="NODE.ITEM"/></param>
  <param id="2" description="Desc.paramOpcional" attribute="out" type="i4">
    <valueReturn/></param>
  <param id="1" description="Desc.paramOpcional" attribute="in" type="i4">
    <valueDefault value="123"/></param>
  <param id="2" description="Desc.paramOpcional" attribute="inout" type="i4">
    <valueItem item="NODO[reg].ITEM"/></param>
  <param id="3" description="Desc.paramOpcional" attribute="inout" type="i4">
    <valueParam param="1"/></param>
  <param id="4" description="Desc.paramOpcional" attribute="in" type="tab">
    <valueNode node="NODE" regFirstIn="2" regLastIn="7" regCountIn="5"
      fillModeOut="append">
      <item id="NomCol1" type="i4" item="ITEM1"/>
      <item id="2" type="i4" item="ITEM2"/>
    </valueNode></param>
  <param id="4" description="Desc.paramOpcional" attribute="in" type="tab">
    <valueTable cols="2" rows="3">
      <th id="NomCol1" type="i4"/>
      <th id="2" type="string"/>
    </tr>
    <valueDefault value="123"/>
    <valueDefault value="123423"/>
  </tr>
  <valueDefault value="123"/>
  <valueDefault value="123423"/>
  </tr>
  <valueDefault value="123"/>
  <valueDefault value="123423"/>
  </tr>
  </valueTable>
</param>
  <extra name="extraMethod2" value=""/>
</method>

```

Meta4Object de conexión

Desde un Meta4Object es posible llamar a los métodos definidos en un archivo XMLDEF. Mediante una herramienta de integración, se puede generar automáticamente el Meta4Object de conexión que tiene como métodos LN4 todos los métodos contenidos en el archivo de mapeo utilizado.

La forma en que la herramienta de integración relaciona los métodos de la aplicación externa con el Meta4Object de conexión se define en el archivo de mapeo XMLMAP correspondiente. El archivo de mapeo relaciona los métodos y parámetros de la aplicación externa con los métodos LN4 y propiedades LN4 del Meta4Object.

El Meta4Object de conexión creado debe tener siempre un nodo raíz llamado M4_GC_INTERFACE, asociado a la estructura de nodo M4_GC_INTERFACE que es común a todos los Meta4Objects. Este nodo contiene un método y una serie de propiedades iguales para todos los Meta4Objects de conexión que se generen.

El método que contiene este nodo es el siguiente:

- **Execute:** ejecuta los métodos específicos de la aplicación externa llamada desde la librería del Conector genérico.

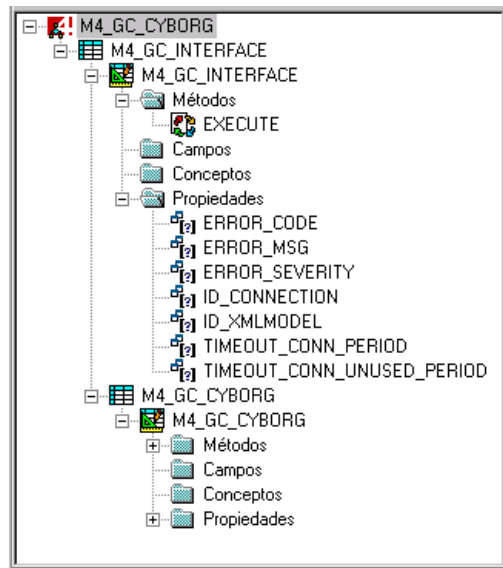
Las propiedades que contiene el nodo son:

- **Error_code:** código de error
- **Error_msg:** mensaje de error
- **Error_severity:** gravedad del error

Al ejecutar un método, la librería de servicio puede devolver un error y decidir si se interrumpe la ejecución de la regla LN4. Estas tres propiedades sólo tienen valor cuando se produce un error. El valor que muestran las propiedades es el que se especifica en la librería de servicio.

- **Id_connection:** identificador de la conexión actual
- **Id_xmlmodel:** identificador del archivo de mapeo utilizado
- **Timeout_conn_period:** tiempo máximo para una conexión activa
- **Timeout_conn_unused_period:** tiempo máximo que una conexión inactiva permanece en la caché

Figura 12. Estructura del Meta4Object de conexión



El resto de los nodos que contiene el Meta4Object son específicos de la aplicación externa. Existen nodos, elementos y métodos asociados a los definidos en el archivo de mapeo. También puede haber nodos relacionados con tablas que enviarán o recibirán los datos de la aplicación externa.

Existe un método LN4 por cada método del archivo de mapeo asociado al Meta4Object. Los parámetros de éste método se corresponderán con:

- Parámetros del método LN4 si la asociación es **ValueParam**
- Elementos de un nodo en el caso de que sea **ValueItem**
- El resultado de la ejecución del método LN4, en el caso de que sea de tipo **ValueReturn**
- Para cada parámetro de tipo tabla, **ValueNode**, existirá un nodo que tendrá tantos elementos como columnas tenga la tabla

Debe ejecutar los métodos que se encuentran en los nodos específicos creados para la aplicación externa.

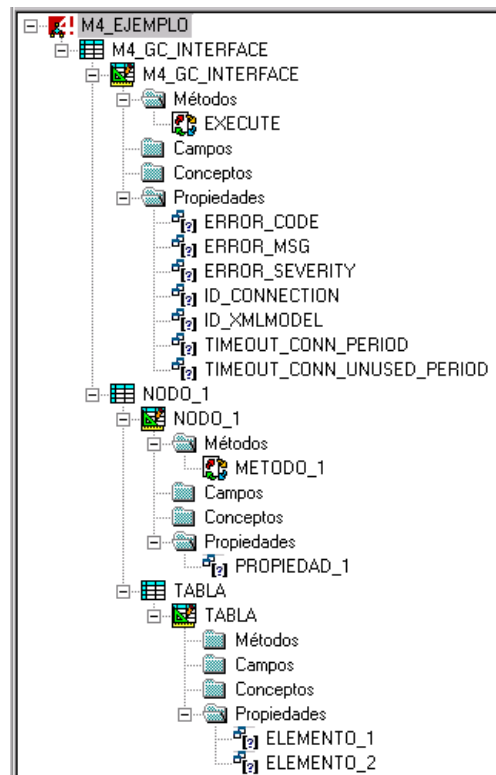
EJEMPLO

El ejemplo a continuación es una sección de un archivo de mapeo, donde se ha definido un método y una serie de parámetros.

```
<method id="doSomething" name="METODO_1" description="Ejemplo de método">
  <param id="1" attribute="in" type="i4"><valueDefault value="123"/>
</param>
  <param id="2" attribute="inout" type="i4"><valueParam param="1"/>
</param>
  <param id="3" attribute="inout" type="i4"><valueItem
item="NODO_1.PROPIEDAD_1"/>
</param>
  <param id="4" attribute="out" type="i4"><valueReturn/>
</param>
  <param id="5" attribute="in" type="tab"><valueNode node="TABLA"
regFirstIn="2" regLastIn="7" fillModeOut="append"><item id="NomCol1"
type="i4" item="ELEMENTO_1"/><item id="2" type="i4"
item="ELEMENTO_2"/></valueNode>
</param>
</method>
```

El archivo de mapeo relaciona el método y parámetros con los métodos LN4 y propiedades LN4 del Meta4Object como se muestra en la siguiente imagen.

Figura 13. Meta4Object de conexión



El resultado de asociar un método de un archivo de mapeo a un Meta4Object de conexión es el siguiente:

- El método se corresponde con el método del Meta4Object METODO_1.
 - El primer parámetro tiene un valor por defecto, en este ejemplo el valor es 123, que va a ser constante en todas las ejecuciones del método.
 - El segundo parámetro definido en el archivo de mapeo, hace que el método LN4 reciba un parámetro LN4. El METODO_1 tiene un único parámetro.
 - El tercer parámetro corresponde a la PROPIEDAD_1.
 - El cuarto parámetro se devolverá como resultado de la ejecución del método LN4.
 - Se crea un nodo para el último parámetro, denominado TABLA con dos elementos que se corresponden con las columnas de la tabla. En este caso ELEMENTO_1 y ELEMENTO_2.
-

Archivo m4ssapi.hpp

```
//=====
//
// (c) Copyright 2004 Meta4 Spain S.A.
// Todos los derechos reservados.
//
// Archivo:          m4ssapi.hpp
// Módulo:          Conector genérico Meta4
// Fecha:           2004.08.27
// Idioma:          C++
// Descripción:     Este módulo define los tipos de datos, estructuras y la
// interfaz para crear DLL de servicio compatibles con el
// Conector genérico Meta4.
// Versión:         6.0
//
//=====

#ifndef __M4SSAPI_HPP__
#define __M4SSAPI_HPP__

// Meta4 usa la implantación ObjectSpace STL.
#if defined(OS_WIN_32) || defined(M4_OBJECTSPACE)
# include <ospace/std/vector>
#else
# include <vector>
#endif
#include <stddef.h>

//=====
// declaración de la clase import/export para win32
//=====
#ifdef _WIN32
# ifdef M4_OBJ_M4SS
# define M4_DECL_M4SS __declspec(dllexport)
# else
# define M4_DECL_M4SS __declspec(dllimport)
# endif
# else
# define M4_DECL_M4SS
# endif

//=====
// Versión de la interfaz DLL
//=====
#define M4_SERVICE_STRUCT_DLL_INTERFACE_VERSION 4.1

//=====
// Tipos predefinidos
//=====
class M4Param ;
class M4TableColumn ;
```

```

/*
 * Vector de columna
 */
typedef vector<M4TableColumn*> M4SSColumnVector ;

/*
 * Tipos de datos
 */
typedef enum {

    m4ssdt_null      = 0,
    m4ssdt_number    = 1,
    m4ssdt_string     = 2,
    m4ssdt_date       = 3,
    m4ssdt_complex    = 4

} M4SSDataType;

/*
 * Tipos de retorno
 */
typedef enum {

    m4ssrt_error      = -1,
    m4ssrt_success     = 0

} M4SSReturnType;

/*
 * Severidad del error
 */
typedef enum {

    m4sses_error       = -1,
    m4sses_info         = 0,
    m4sses_warning      = 1

} M4SSErrorSeverity;

/*
 * Modos de parámetros
 */
typedef enum {

    m4sspm_in           = 1,
    m4sspm_out          = 2,
    m4sspm_inout        = 3,
    m4sspm_ret          = 4,
    m4sspm_property     = 5

} M4SSParameterMode;

/*
 * Tipos de parámetros
 */
typedef enum {

    m4sspt_tab          = 1,          // table (array of struct)
    m4sspt_str          = 2,          // string
    m4sspt_bin          = 3,          // binary

```

```

m4sspt_fil      = 4,      // file
m4sspt_dat      = 5,      // date
m4sspt_tim      = 6,      // time
m4sspt_i1       = 7,      // integer 1
m4sspt_i2       = 8,      // integer 2
m4sspt_i4       = 9,      // integer 4
m4sspt_i8       = 10,     // integer 8
m4sspt_u1       = 11,     // unsigned 1
m4sspt_u2       = 12,     // unsigned 2
m4sspt_u4       = 13,     // unsigned 4
m4sspt_u8       = 14,     // unsigned 8
m4sspt_flo      = 15,     // float
m4sspt_dou      = 16,     // double
m4sspt_booo     = 17,     // boolean
m4sspt_array    = 18,     // array
m4sspt_struct   = 19      // structure

} M4SSParameterType;

//=====
// Estructuras de datos predefinidas
//=====

/*
 * Variant
 */
typedef struct {

    M4SSDataType    Type ;
    union {

        double      Number ;
        char*       String ;

        struct {

            short    Year ;
            char     Month ;
            char     Day ;
            char     Hour ;
            char     Minute ;
            char     Second ;

        }           Date ;

        M4Param*    Complex ;

    } Data ;

} M4SSVariant;

//=====
// Interfaces exportadas
//=====

/*
 * Esta clase define la interfaz para parámetros de metadatos de

```

```

    * información.
    */
class M4_DECL_M4SS M4Param
{
public:

    virtual                    ~M4Param( void ) {} ;

    virtual const char*        GetIdentifier( void ) const = 0 ;
    virtual M4SSParameterMode  GetMode( void ) const = 0 ;
    virtual M4SSParameterType  GetType( void ) const = 0 ;
    virtual const char*        GetNativeName( void ) const = 0 ;

    virtual size_t             NumberOfExtraInfo( void ) const = 0 ;
    virtual M4SSReturnType     GetExtraInfo( size_t ai_iIndex, const char**
ao_ppccName, const M4SSVariant** ao_ppcvValue ) const = 0 ;
};

/*
 * Esta clase define la interfaz para parámetros simples de
 * datos de información.
 */
class M4_DECL_M4SS M4SimpleParam : public M4Param
{
public:

    virtual                    ~M4SimpleParam( void ) {} ;

    virtual const M4SSVariant*  GetValue( void ) const = 0 ;
    virtual void                SetValue( const M4SSVariant* ai_pcvValue ) = 0 ;
};

/*
 * Esta clase define la interfaz para columnas de tabla de
 * metadatos de información.
 */
class M4_DECL_M4SS M4TableColumn
{
public:

    virtual                    ~M4TableColumn( void ) {} ;

    virtual const char*        GetIdentifier( void ) const = 0 ;
    virtual M4SSParameterType  GetType( void ) const = 0 ;
    virtual const char*        GetNativeName( void ) const = 0 ;
    virtual const M4SSColumnVector*  GetChildColumns( void ) const = 0 ;
};

/*
 * Esta clase define la interfaz para parámetros de tabla de
 * datos de información.
 */
class M4_DECL_M4SS M4TableParam : public M4Param
{
public:

    virtual                    ~M4TableParam( void ) {} ;

```

```

        virtual size_t      NumberOfColumns( void ) const = 0 ;
        virtual M4SSReturnType GetColumn( size_t ai_iIndex, const M4TableColumn**
ao_ppcoColumn ) const = 0 ;

        virtual size_t      NumberOfRows( void ) const = 0 ;
        virtual M4SSReturnType AddRow( void ) = 0 ;
        virtual M4SSReturnType RemoveRow( size_t ai_iRow ) = 0 ;
        virtual void         RemoveAllRows( void ) = 0 ;

        virtual M4SSReturnType GetValue( size_t ai_iRow, size_t ai_iColumn, const
M4SSVariant** ao_ppcvValue ) const = 0 ;
        virtual M4SSReturnType SetValue( size_t ai_iRow, size_t ai_iColumn, const
M4SSVariant* ai_pcvValue ) = 0 ;
    };

/*
 * Esta clase define la interfaz para pasar argumentos.
 */
class M4_DECL_M4SS M4ServiceStruct
{
public:

        virtual ~M4ServiceStruct( void ) {} ;

        virtual const char* GetMethodId( void ) const = 0 ;
        virtual const char* GetLibraryId( void ) const = 0 ;
        virtual const char* GetMethodNativeName( void ) const = 0 ;

        virtual void SetErrorCode( int ai_iCode ) = 0 ;
        virtual void SetErrorSeverity( M4SSErrorSeverity ai_iSeverity )
= 0 ;
        virtual void SetErrorMsg( const char* ai_pccMessage ) = 0 ;
        virtual void SetErrorMsgFormat( const char* ai_pccFormat, ... )
= 0 ;

        virtual size_t      NumberOfParams( void ) const = 0 ;
        virtual M4Param* GetParam( size_t ai_iIndex ) const = 0 ;
        virtual M4Param* FindParamById( const char* ai_pccIdentifier ) const
= 0 ;

        virtual size_t      NumberOfProperties( void ) const = 0 ;
        virtual M4Param* GetProperty( size_t ai_iIndex ) const = 0 ;

        virtual size_t      NumberOfMethodExtraInfo( void ) const = 0 ;
        virtual M4SSReturnType GetMethodExtraInfo( size_t ai_iIndex, const char**
ao_ppccName, const M4SSVariant** ao_ppcvValue ) const = 0 ;

        virtual size_t      NumberOfLibraryExtraInfo( void ) const = 0 ;
        virtual M4SSReturnType GetLibraryExtraInfo( size_t ai_iIndex, const
char** ao_ppccName, const M4SSVariant** ao_ppcvValue ) const = 0 ;
    };

/*
 * Esta clase define la interfaz para la conexión.
 */
class M4_DECL_M4SS M4ServiceConnection
{
public:

```

```

        virtual                ~M4ServiceConnection( void ) {} ;

        virtual M4SSReturnTypes Reset( void ) = 0 ;
        virtual const char*      GetHash( void ) const = 0 ;
        virtual void              SetHash( const char* ai_pccHash ) = 0 ;
    } ;

    /*
     * Esta clase define la interfaz para la conexión colectiva.
     */
    class M4_DECL_M4SS M4GenericServiceConnection
    {
    public:

        virtual                ~M4GenericServiceConnection( void ) {} ;

        virtual void            NotifyReset( void ) = 0 ;
        virtual void            NotifyLogoff( void ) = 0 ;

        virtual M4SSReturnTypes LookUpServiceConnection( const char*
ai_pccHash ) = 0 ;
        virtual M4ServiceConnection* GetServiceConnection( void ) = 0 ;
        virtual void            SetServiceConnection( M4ServiceConnection*
ai_poServiceConnection ) = 0 ;
    };

    //=====
    // Clases exportadas
    //=====

    /*
     * Esta clase define la implantación por defecto para la conexión.
     */
    class M4_DECL_M4SS M4DefaultServiceConnection : public M4ServiceConnection
    {
    public:

        M4DefaultServiceConnection( const char* ai_pccHash ) ;
        virtual                ~M4DefaultServiceConnection( void ) ;

        const char*            GetHash( void ) const ;
        void                    SetHash( const char* ai_pccHash ) ;

    protected:

        char*                  m_pccHash ;
    };

    //=====
    // Prototipos para funciones importadas por la dll del servicio.
    // Estas funciones se declaran aquí, pero se definen en la dll de servicio
    // del usuario
    //=====

    #ifndef M4_OBJ_M4SS

```

```
#ifndef _WIN32
# define M4_DECL_GC_SVC_DLL extern "C" __declspec(dllexport)
#else
# define M4_DECL_GC_SVC_DLL extern "C"
#endif

/*
 * Función de versión de la interfaz. El cuerpo de esta función debe ser
 * siempre:
 *   double GetInterfaceVersion(void)
 *   {
 *       return (M4_SERVICE_STRUCT_DLL_INTERFACE_VERSION);
 *   }
 */
M4_DECL_GC_SVC_DLL double GetInterfaceVersion( void ) ;

/*
 * Punto de entrada de la DLL de servicio
 */
M4_DECL_GC_SVC_DLL M4SSReturnType Execute( M4ServiceStruct*
aio_poServiceStruct, M4GenericServiceConnection *ai_poConnection ) ;

#endif // M4_OBJ_M4SS

#endif // __M4SSAPI_HPP__
```